ECE 491 Image Processing and Computer Vision (Fall 2026)



Lecture 21 Machine Learning: Intro and Neural Nets

Instructor: Tianfu (Matt) Wu, Associate Professor in ECE

PI of the iVMCL lab (interpretable Visual Modeling, Computing and Learning)

Website: https://research.ece.ncsu.edu/ivmcl/

Code & Models: https://github.com/iVMCL/

NC STATE UNIVERSITY Electrical and Computer Engineering

Outline

- **Topics:** Why learn instead of hand-code; logistic and softmax regression; gradient descent; from one layer to deep networks; backpropagation.

- **Key Concept:** Learning = pick a model, pick a loss, follow the gradient. Backprop computes that gradient for any network (**Encoder-Decoder** view: w = Dec(Enc(x))).

A Thought Experiment

- How shall we teach a computer to solve a specific problem?
  - Consider **image classification**: a core task and technique in CV



This image by Nikita is licensed under CC-BY 2.0

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}



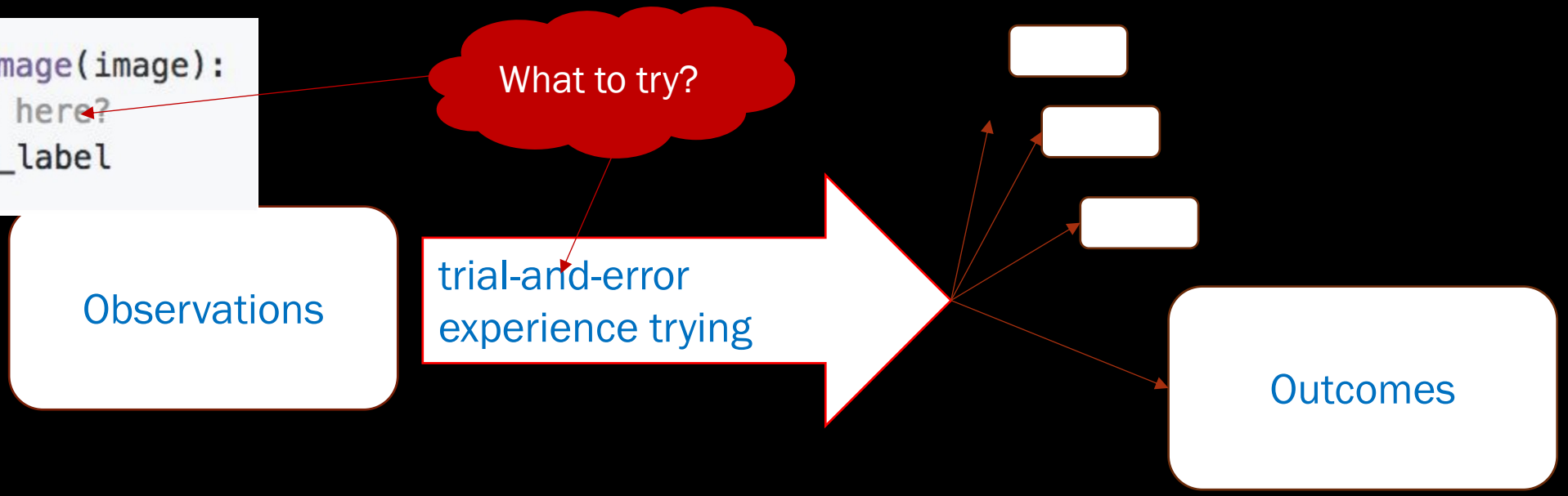
cat

Image Source: http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture2.pdf

A Thought Experiment

- How shall we teach a computer to solve a specific problem?
  - Consider **image classification**: a core task and technique in CV
  - **Learning based systems:** not directly programmed to solve a problem, instead develop own program based on:
    - Examples (observations) of how they should behave
    - From trial-and-error experience trying to solve the problem

```
def classify_image(image):
  # Some magic here?
  return class_label
```



Challenges: Viewpoint

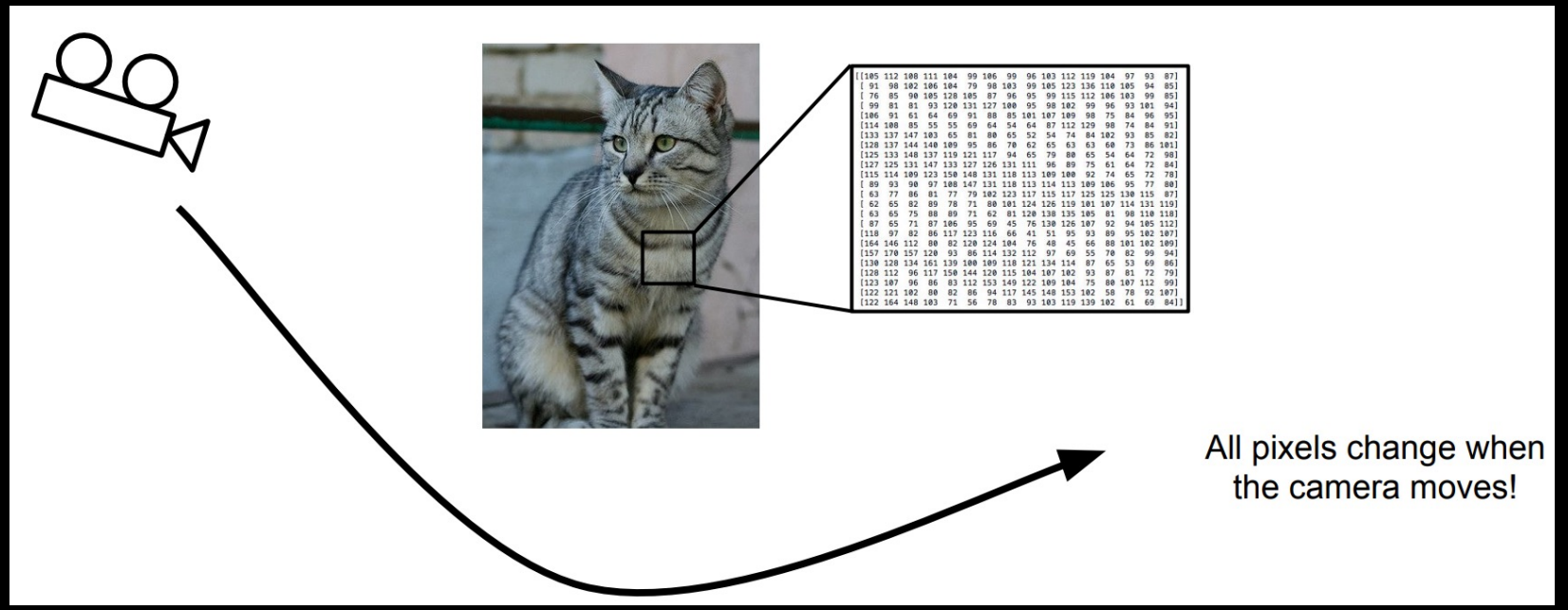


Image Source: http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture2.pdf

Challenges: Illumination

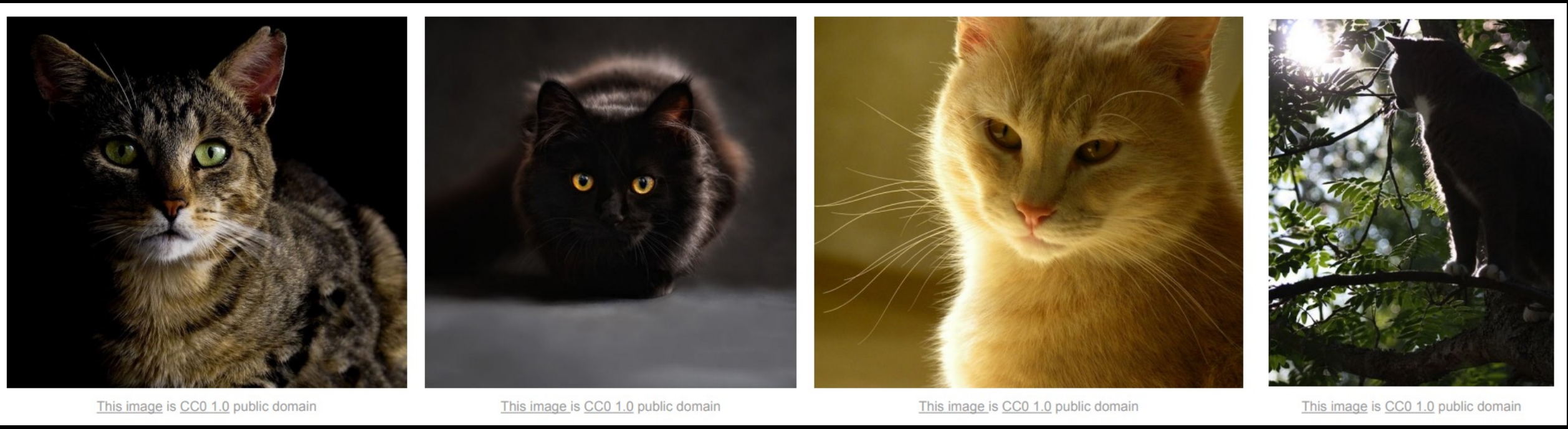
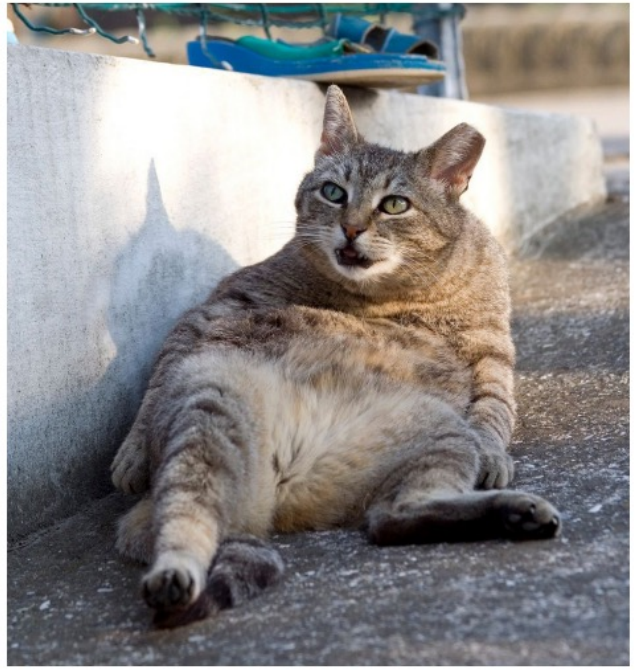
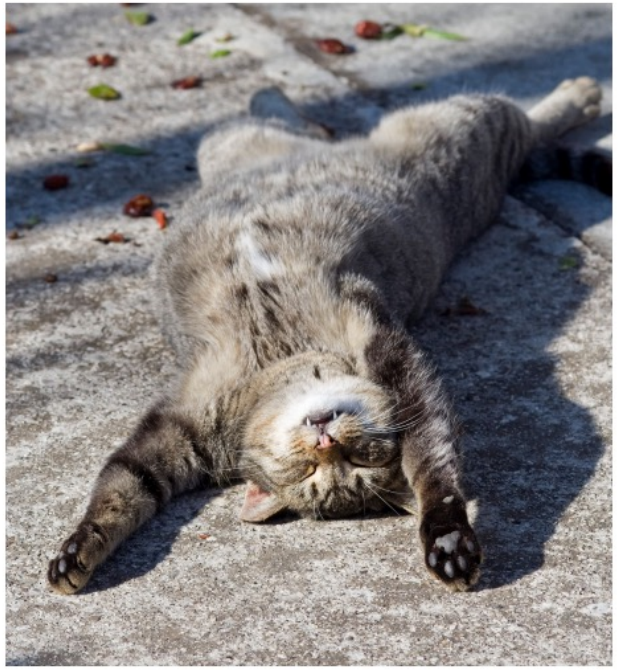


Image Source: http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture2.pdf

Challenges: Deformation



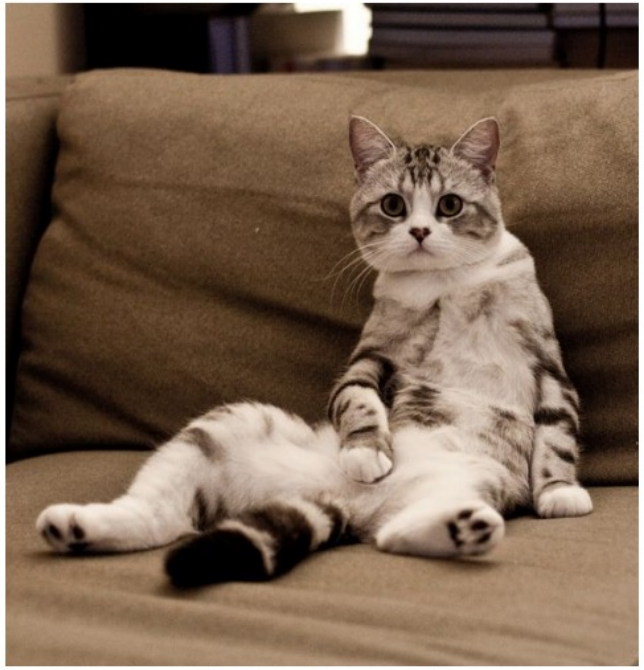
This image by Umberto Salvagnin is licensed under CC-BY 2.0



This image by Umberto Salvagnin is licensed under CC-BY 2.0



This image by sare bear is licensed under CC-BY 2.0



This image by Tom Thai is licensed under CC-BY 2.0

Image Source: http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture2.pdf

Challenges: Occlusion



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain

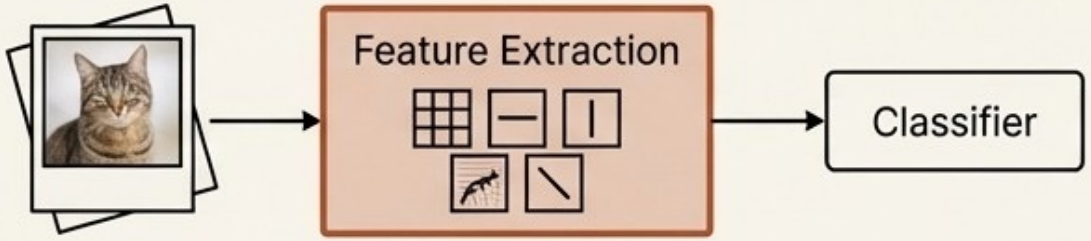


This image by jonsson is licensed under CC-BY 2.0

Image Source: http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture2.pdf

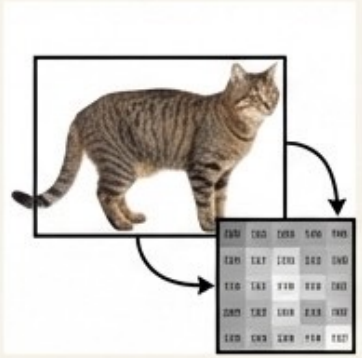
The Old Tools: Hand-Crafting the Rules of Vision

For decades, vision systems relied on hand-crafted features. Experts would design specific filters to detect edges, corners, textures, and colors. This approach was brittle and struggled with the immense variability of the real world.



- **Viewpoint**: Objects look different from various angles.
- **Illumination**: Lighting changes an object's appearance entirely.
- **Deformation & Occlusion**: Objects can be bent, twisted, or partially hidden.
- **Clutter & Intraclass Variation**: The same object can appear in messy scenes or come in countless variations (e.g., different cat breeds).

Viewpoint



Illumination



This image is CC0 1.0 public domain

Deformation



This image by Umberto Salvagnin is licensed under CC-BY 2.0

Occlusion



This image by jonsson is licensed under CC-BY 2.0

Clutter



This image is CC0 1.0 public domain

Intraclass Variation

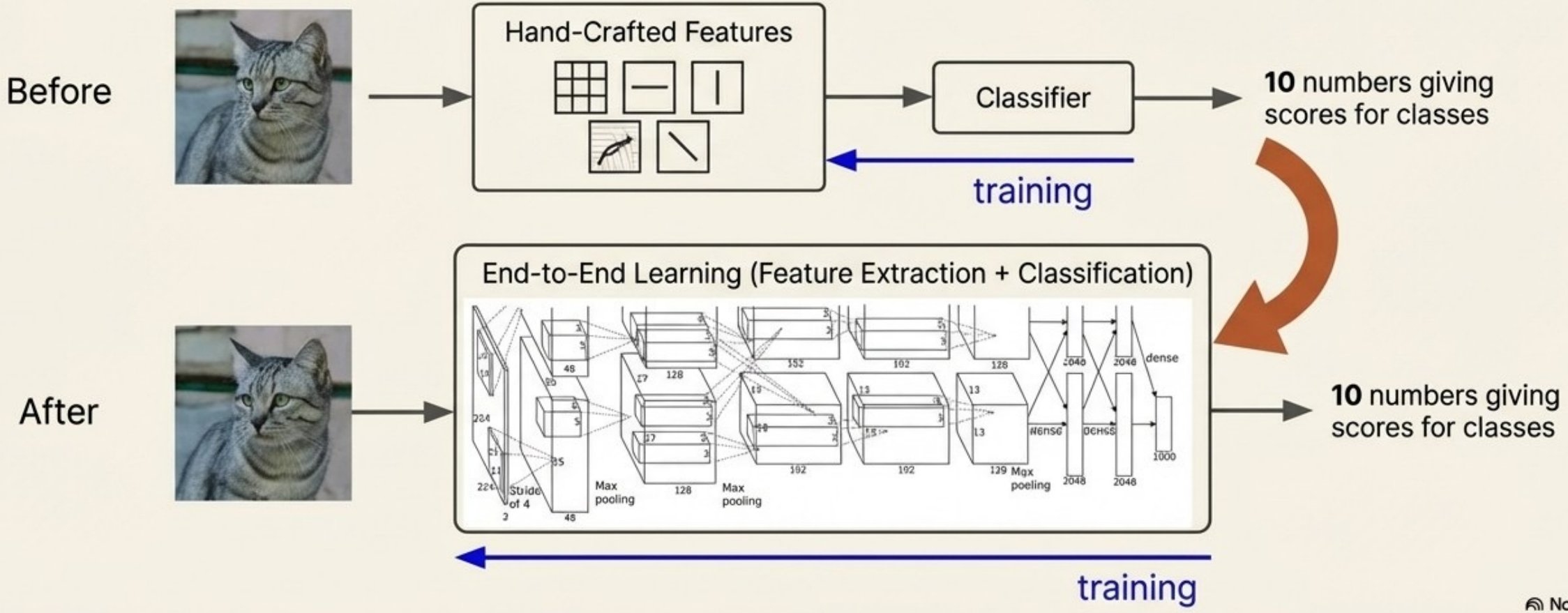


This image is CC0 1.0 public domain

NotebookLM

The Revolution: From Hand-Crafted Features to Learned Representations

The deep learning revolution absorbed the feature extraction step directly into the model. Instead of telling the machine *what* to look for (edges, corners), we provide it with raw data and let it learn the optimal representations for a given task. This is 'end-to-end' learning. Representation is everything.

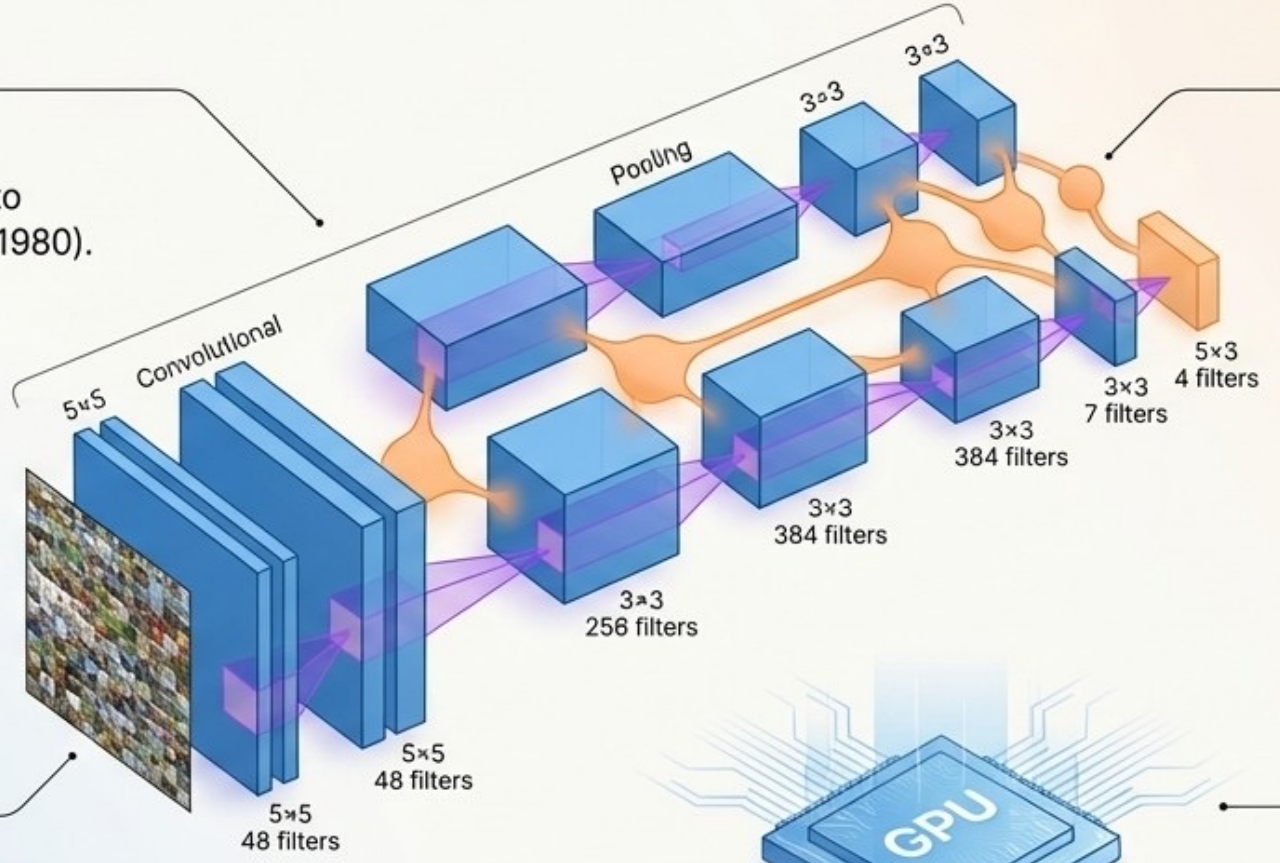


NotebookLM

The Winning Formula: A Perfect Storm of Converging Ideas

The Architecture

A deep convolutional neural network, structurally similar to Fukushima's **Neocognitron** (1980).



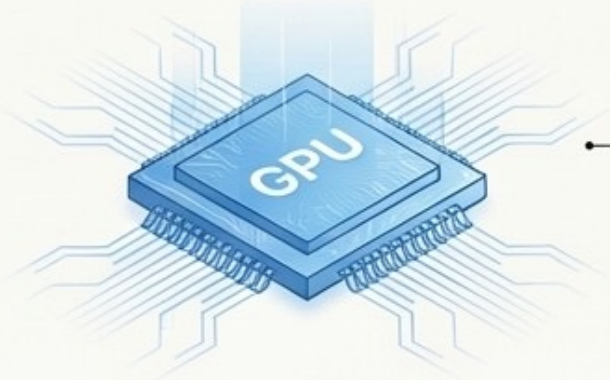
The Learning Algorithm

Trained efficiently using Hinton et al.'s **Backpropagation** (1986).



The Fuel

The massive ImageNet dataset provided the necessary data for the network to learn robust features.

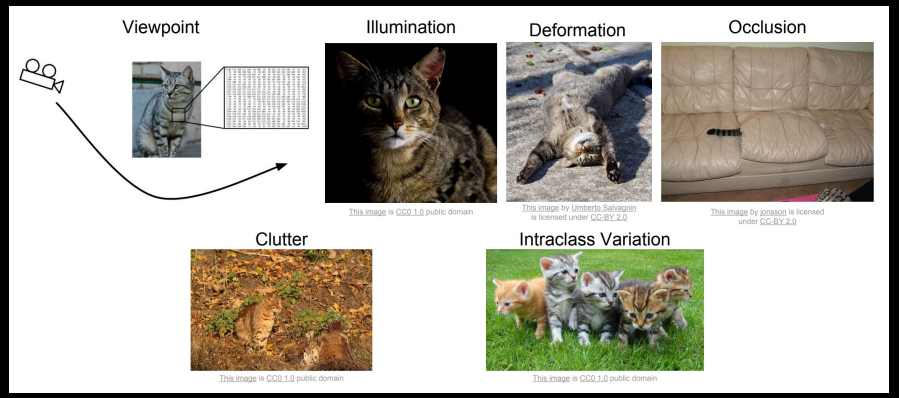


The Engine

Modern GPUs provided the parallel computation power necessary to train such a large model in a reasonable time.

NotebookLM

A Generic Setup: Encoder-Decoder



$X = x$ (e.g., image)

$$Enc(x) \in R^D \longrightarrow w = Dec(Enc(x))$$

w (world state of x, e.g., image label)

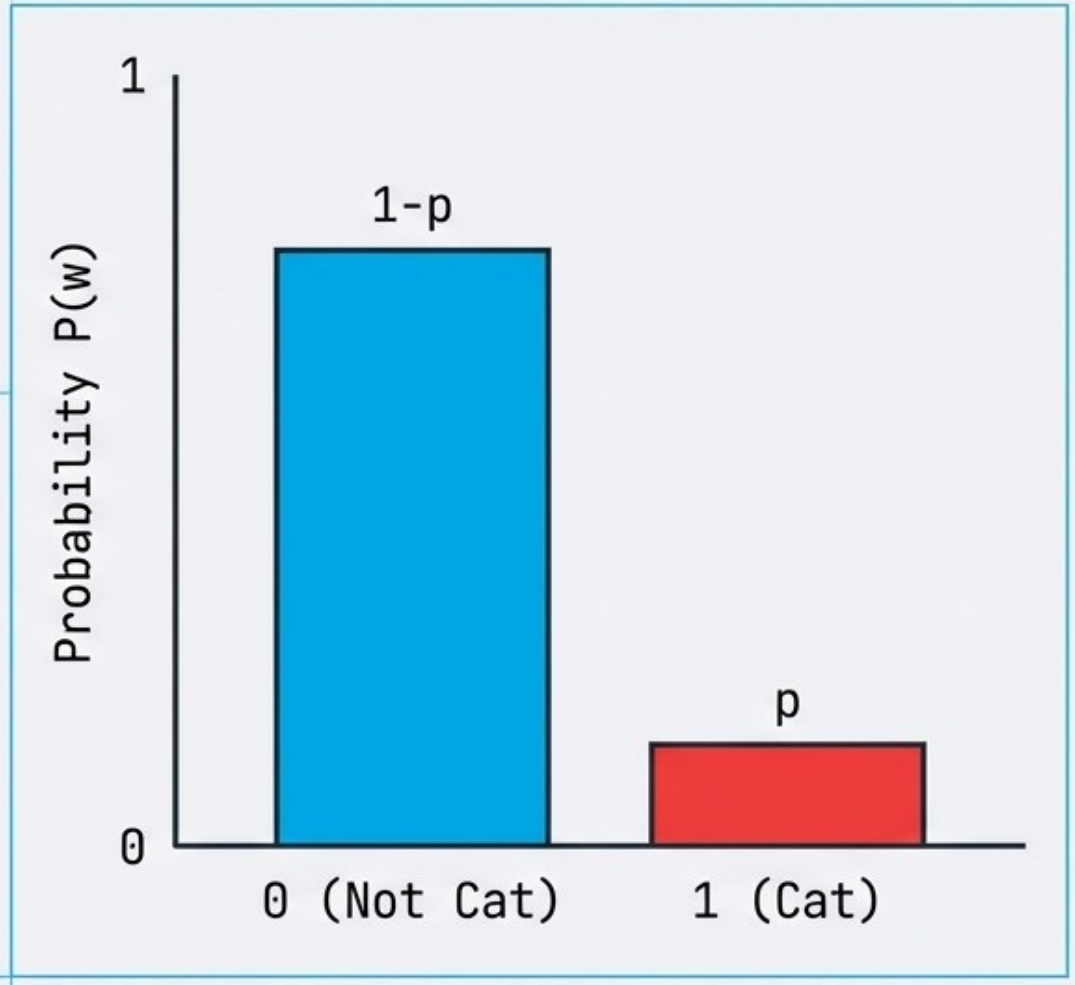
```
def classify_image(image):
  # Some magic here?
  return class_label
```

```
def Encoder(image):
    # Some magic transformation?
    # stretching and squashing data points
    feature = Transformation(image)
    return feature

def Decoder(feature):
    # Some not-so-magic prediction :)
    world_state = Prediction(feature)
    return world_state

def classify_image(image):
    # some magic here?
    feature = Encoder(image)
    class_label = Decoder(feature)
    return class_label
```

Binary Classification via Logistic Regression



The Task:

Determine if input belongs to Class 1 or Class 0.

The Assumption:

World State W follows a Bernoulli distribution.

W ~ Bernoulli(p)

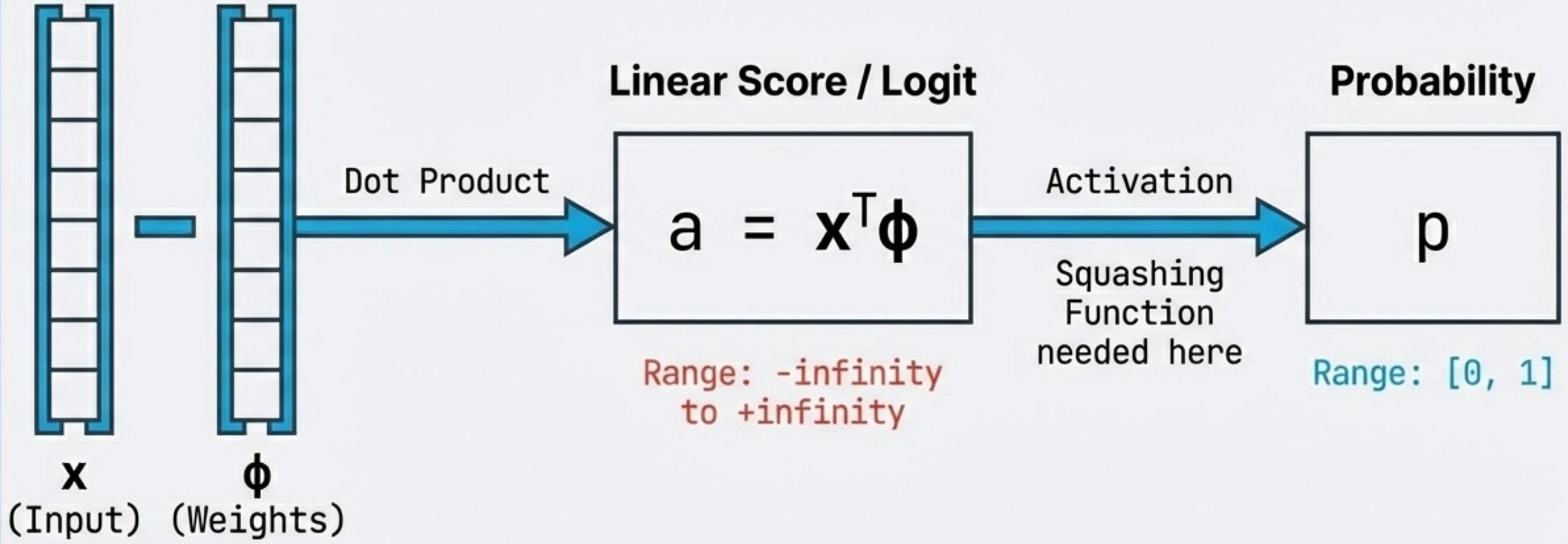
The Goal:

Predict the single parameter 'p' (probability of success) given the input 'x'.

p = P(W=1 | X)

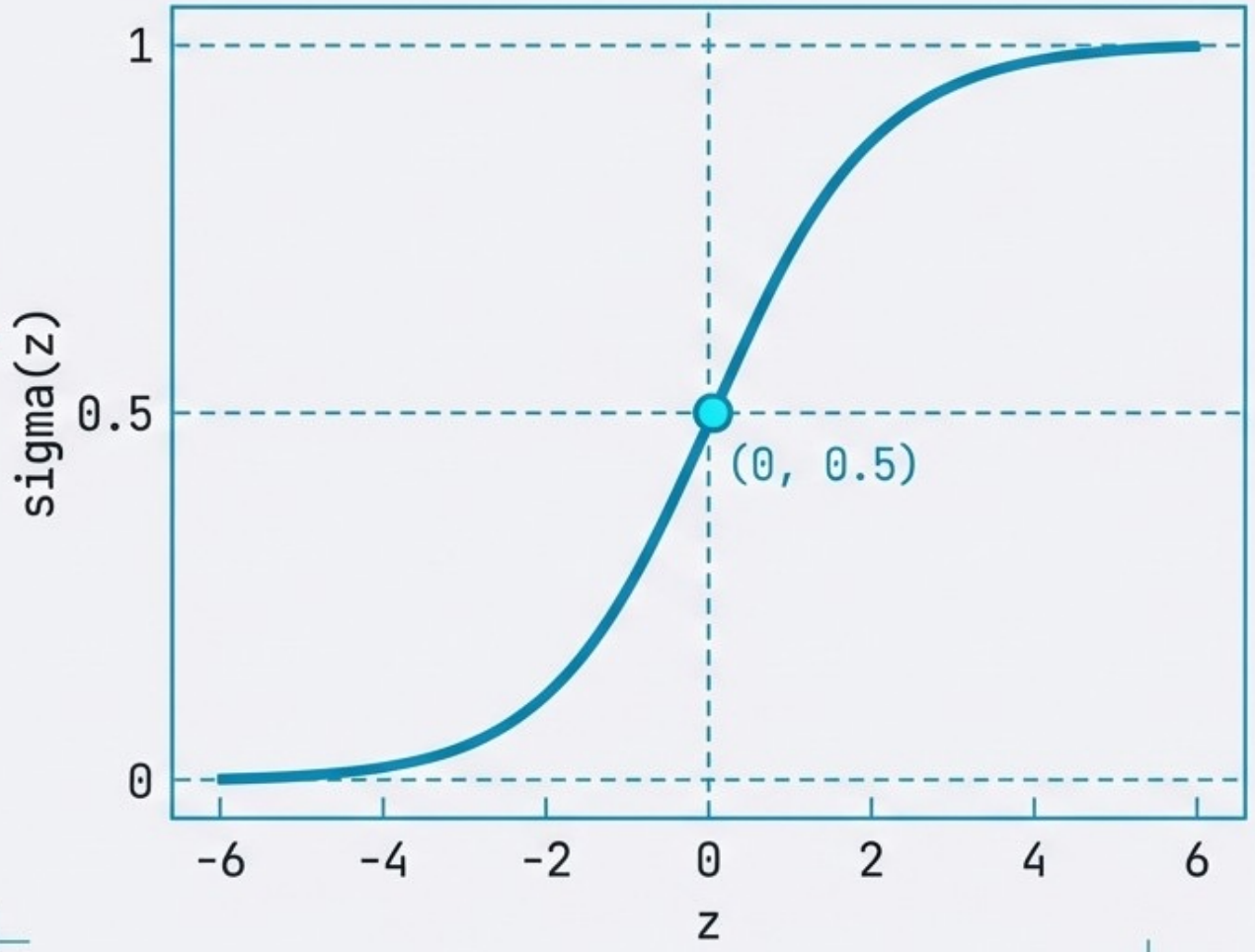
NotebookLM

The Mechanics: From Linear Score to Probability



NotebookLM

The Sigmoid Function

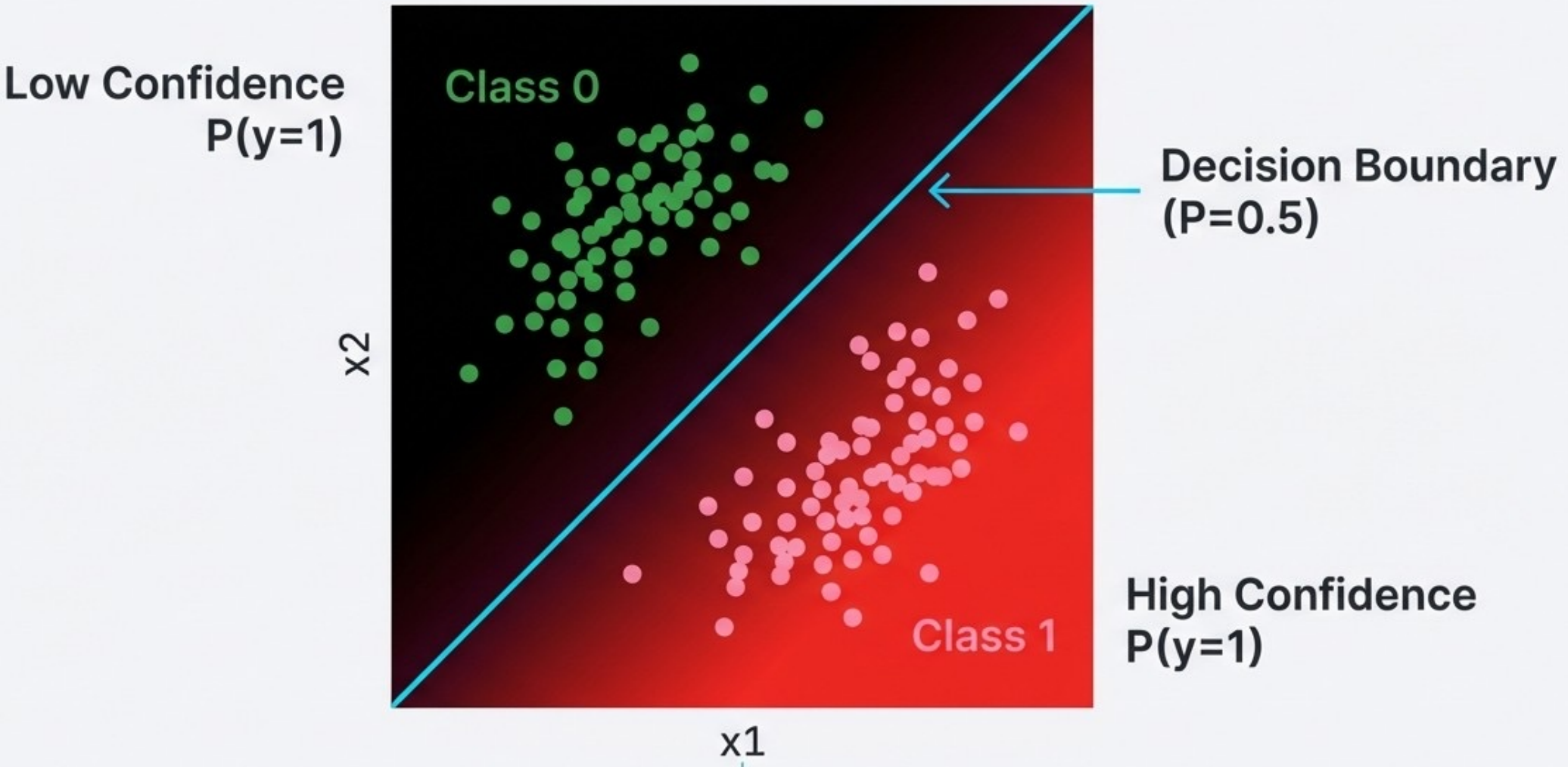


$$\text{sigma}(z) = \frac{1}{1 + e^{-z}}$$

- Maps any real number to [0, 1]
- Interpretable as probability
- Differentiable (crucial for training)

NotebookLM

Visualizing the Decision Boundary



NotebookLM

Estimation via Optimization (MLE)

How do we choose the best weights (ϕ)?
We find the parameters that maximize the probability of the observed labels.

$$\text{Likelihood } L(\phi) = \prod P(w_i \mid x_i; \phi)$$



$$\text{Log-Likelihood} = \sum \log(\, P(w_i \mid x_i; \phi))$$

Log transformation turns products into sums to prevent numerical underflow.

NotebookLM

The Loss Function: Negative Log-Likelihood



In Machine Learning, we minimize "Badness" (Loss) rather than maximizing "Goodness".

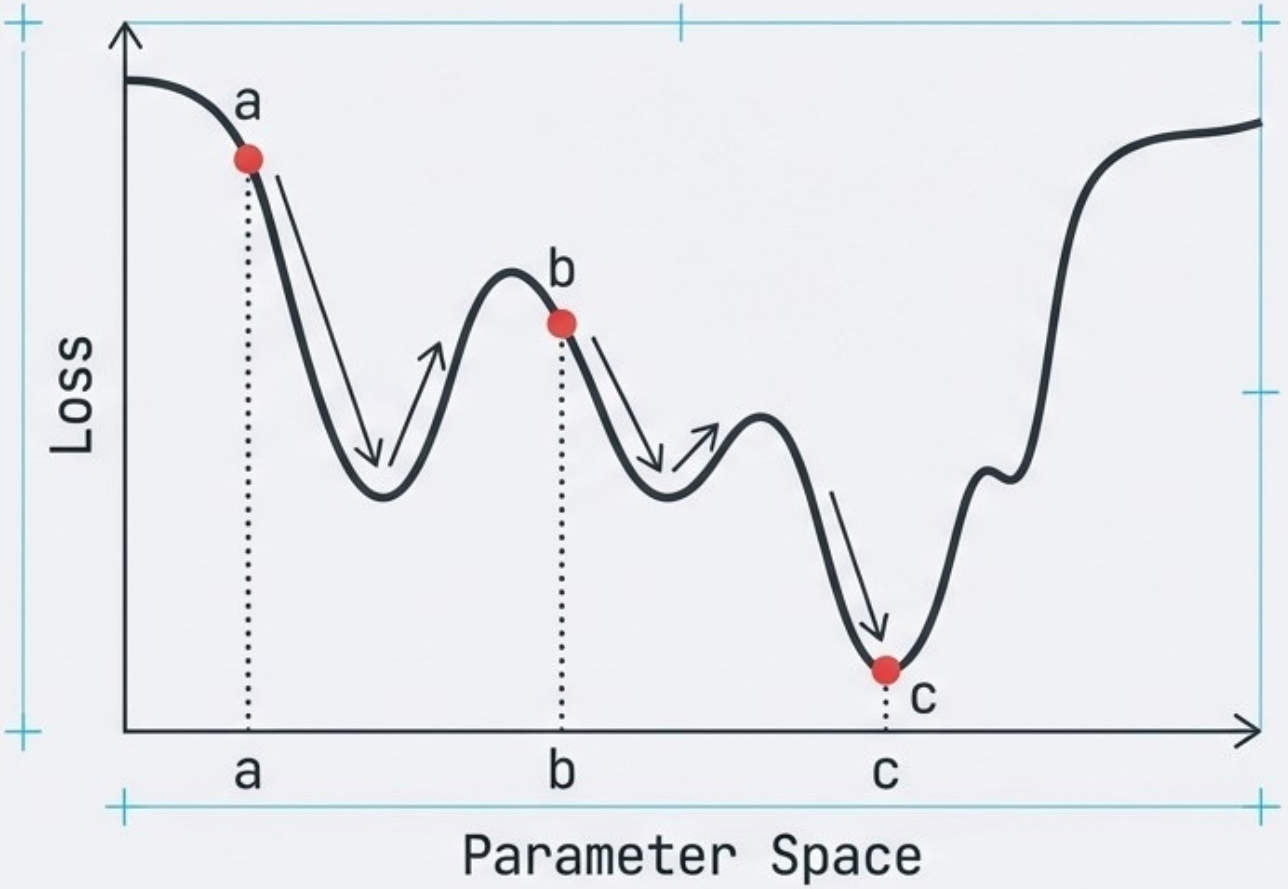
$$\text{Loss}(\phi) = -\sum [w \log(p) + (1-w) \log(1-p)]$$

Key Insight:

Convexity:
This function has a single global minimum. Optimization is guaranteed to find the best solution.

NotebookLM

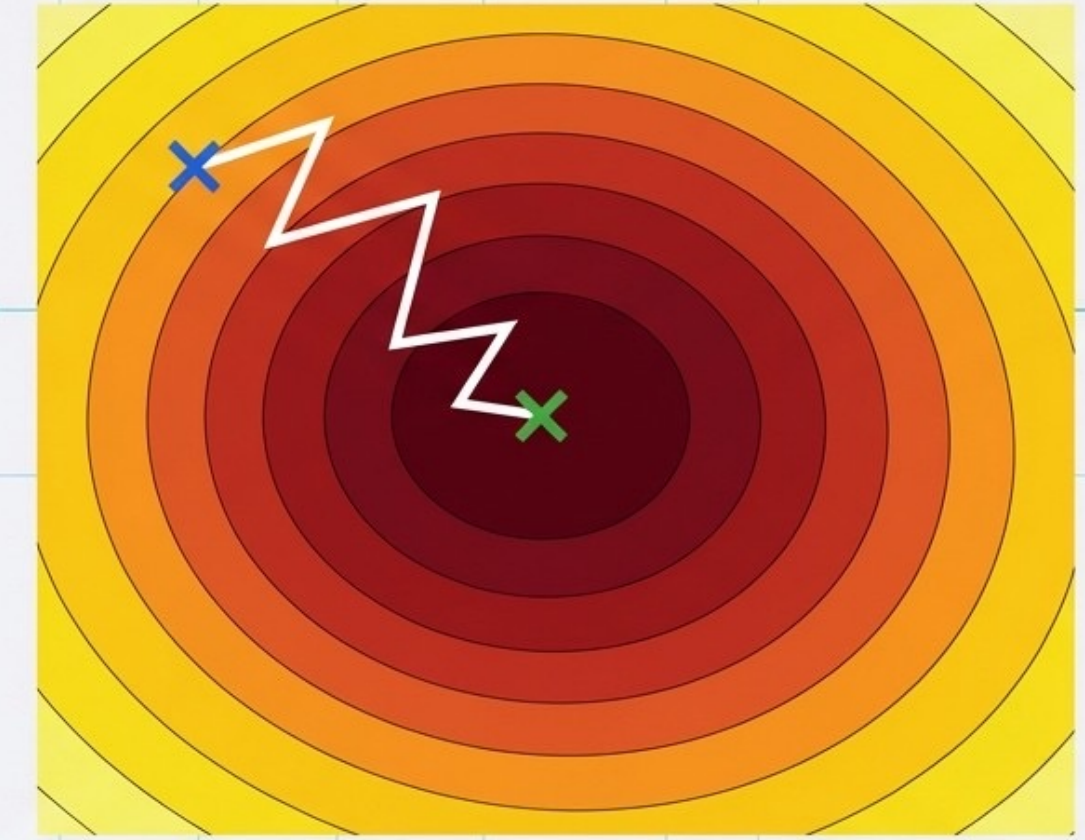
Navigation Strategy: Iterative Optimization



1. Start with random estimate.
2. Choose search direction (Gradient).
3. Step size (Learning Rate).
4. Repeat until cost stops decreasing.

NotebookLM

First Order: Gradient Descent



Strategy: Look at the slope at your feet. Step in steepest downhill direction.

Pros: Computationally cheap.

Cons: "Zig-zagging" behavior. Slow convergence near the optimum.

NotebookLM

Summary: From Pixels to Probability



Semantic Gap
Mapping Data (X) to World (W).



Logistic Model
Linear score squashed into probability.



Loss Function
Minimizing Negative Log-Likelihood.



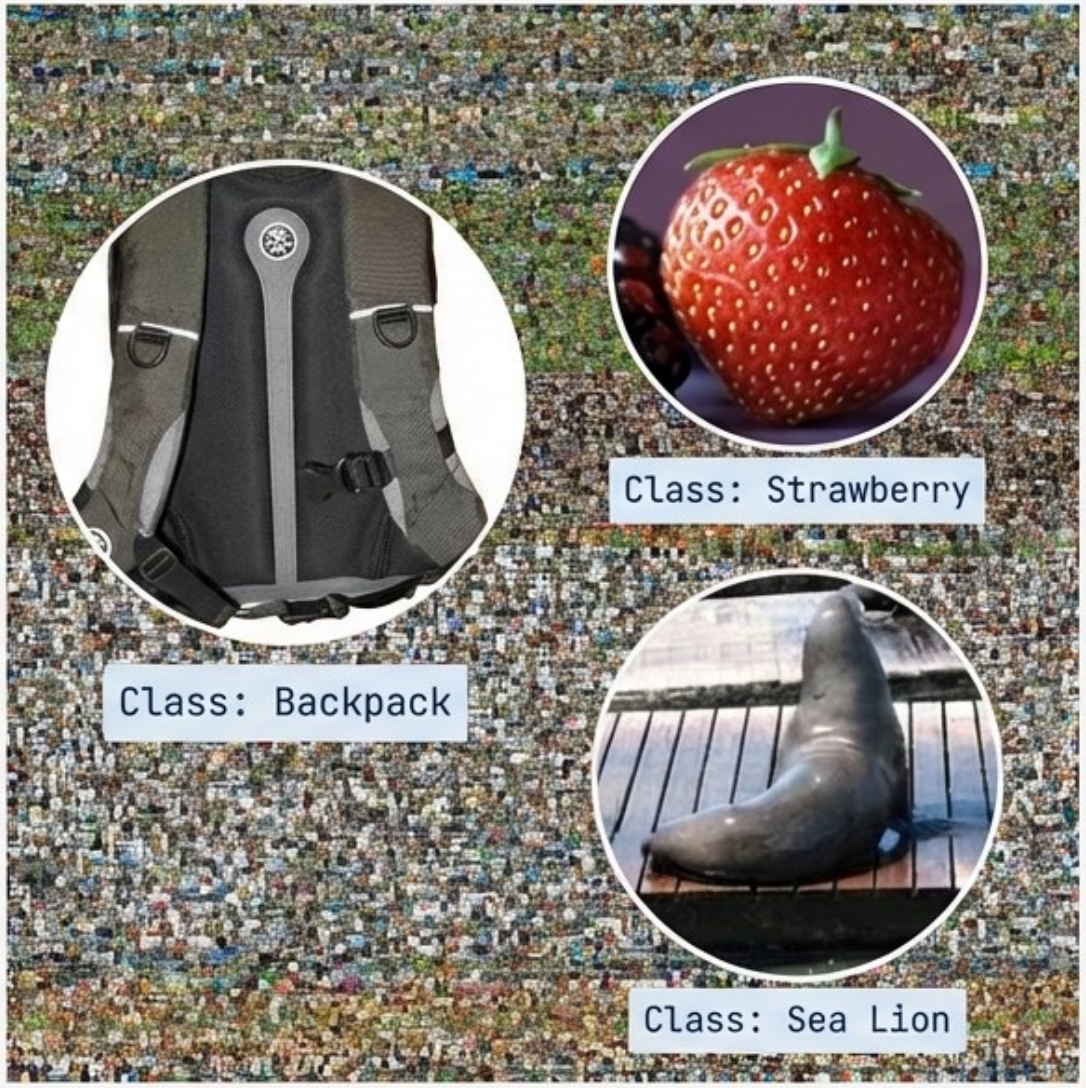
Optimization
Navigating the landscape via Gradient Descent.

This statistical framework forms the baseline for Deep Learning; Neural Networks are essentially stacked layers of these linear classifiers.

NotebookLM

The Challenge: Scaling to N-Classes

- **Binary Limitations:** Logistic Regression works for Yes/No (2-class) problems but fails to scale elegantly to the real world.
- **The ImageNet Scale:** Real-world recognition requires handling thousands of distinct categories simultaneously.
- **The Goal:** Move from ‘One-vs-All’ binary classifiers to a native Categorical Distribution.



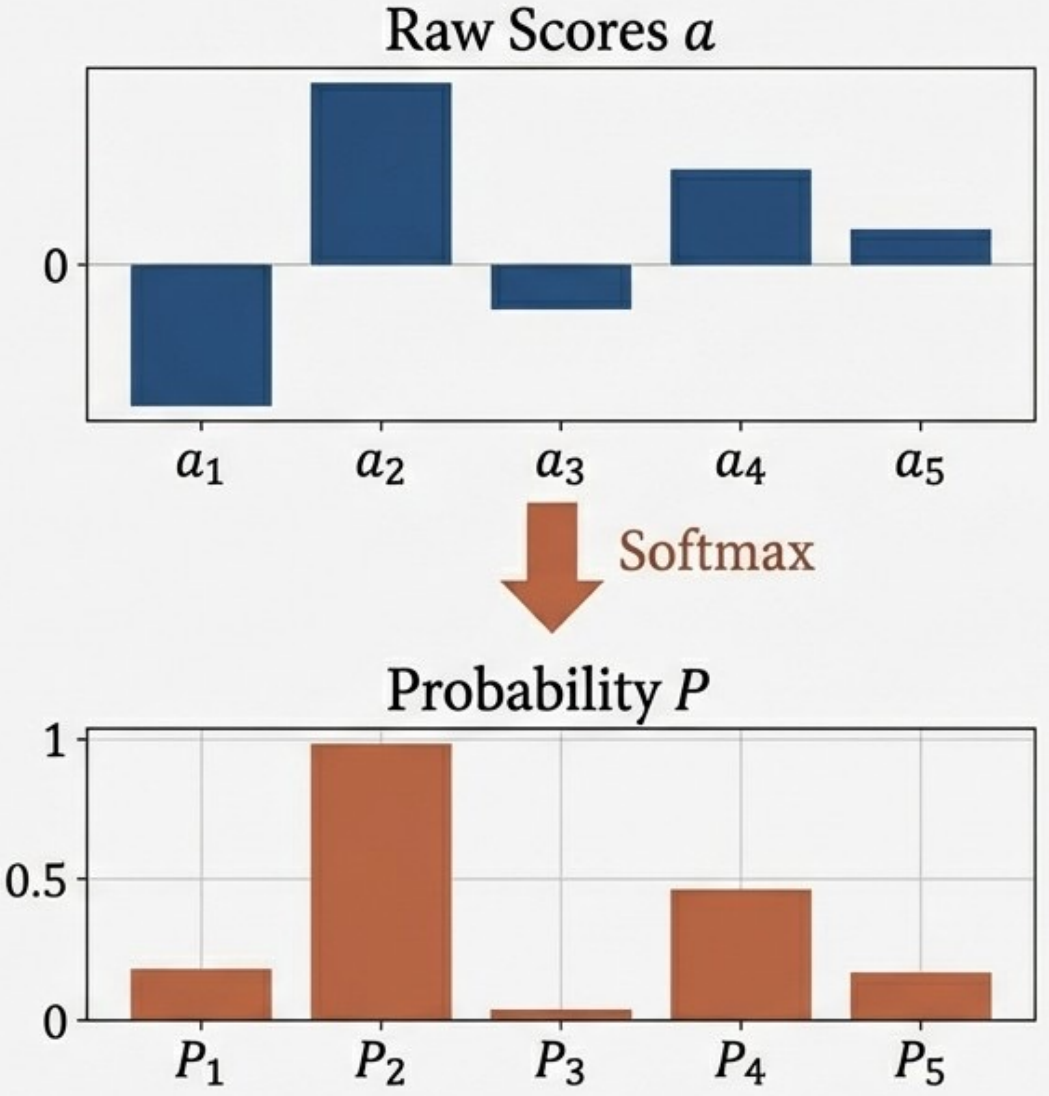
NotebookLM

The Model: Softmax Regression

- **Definition:** A model assuming a Categorical distribution over the world state.
- **The Process:**
  1. Linear Score: Map input x to activation scores a_c using weights θ.

$$P(y = c|x) = \text{Softmax}(a)_c = \frac{e^{a_c}}{\sum_{j=1}^{C} e^{a_j}}$$

  2. Normalization: Exponentiate and divide by the sum to get probabilities.



NotebookLM

The Objective: Maximum Likelihood Estimation (MLE)

How do we choose the parameters θ?

The Goal: Maximize the probability of the correct labels in the training data.

$$L(\theta) = \prod_{i=1}^{N} P(y_i \mid x_i; \theta)$$

↓

The Problem: Multiplying many small probabilities leads to numerical underflow.

↓

The Solution: Maximize the Log-Likelihood instead (summation is more stable than products).

$$L(\theta) = \sum_{i=1}^{N} \log\left(P(y_{i,\text{true}} \mid x_i)\right)$$

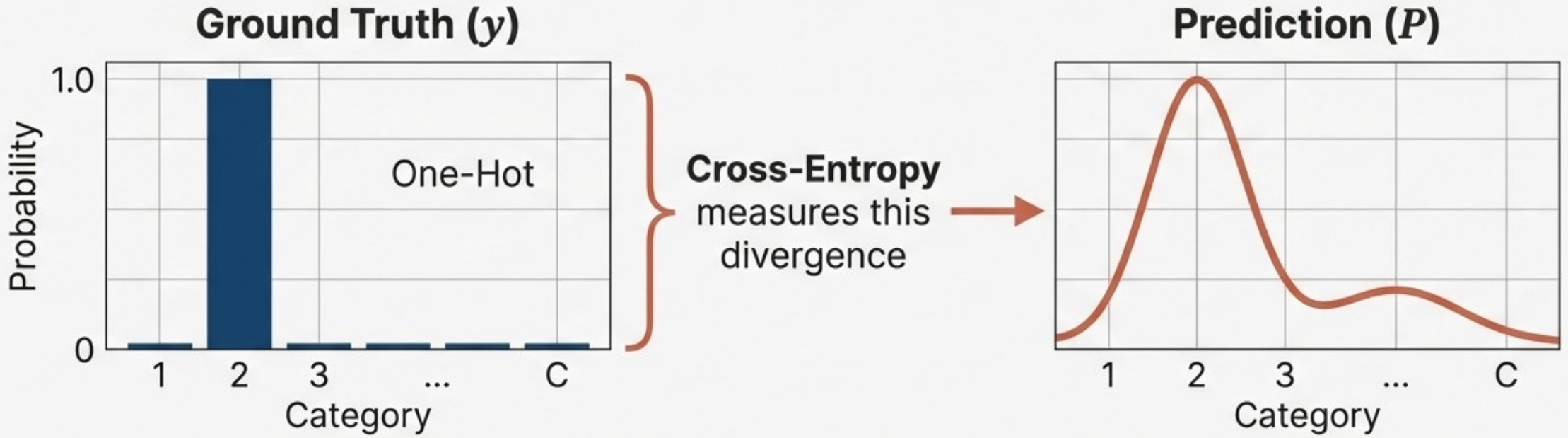
← **Note:** There is no closed-form solution. We must find θ via iterative optimization.

NotebookLM

The Cost Function: Cross-Entropy

Maximizing Log-Likelihood is mathematically equivalent to minimizing the Cross-Entropy loss.

$$\xi(\theta) = -\sum_{c=1}^{C} y_c \cdot \log(P_c)$$



NotebookLM

The Mechanism: Gradient Descent

Deriving the update rule.

The Calculus

$$\frac{\partial \xi}{\partial a} = -\sum y \frac{\partial \log P}{\partial a}$$

Gradient of Cross-Entropy loss w.r.t. activation scores (a).

The Expansion

$$\ldots = P_i - y_i$$

Substituting Softmax derivative simplifies the term.

The Result

Gradient = Prediction (P) - Ground Truth (Y)

This simple difference drives the parameter updates: if Prediction > Truth, push weights down. If Prediction < Truth, push weights up.

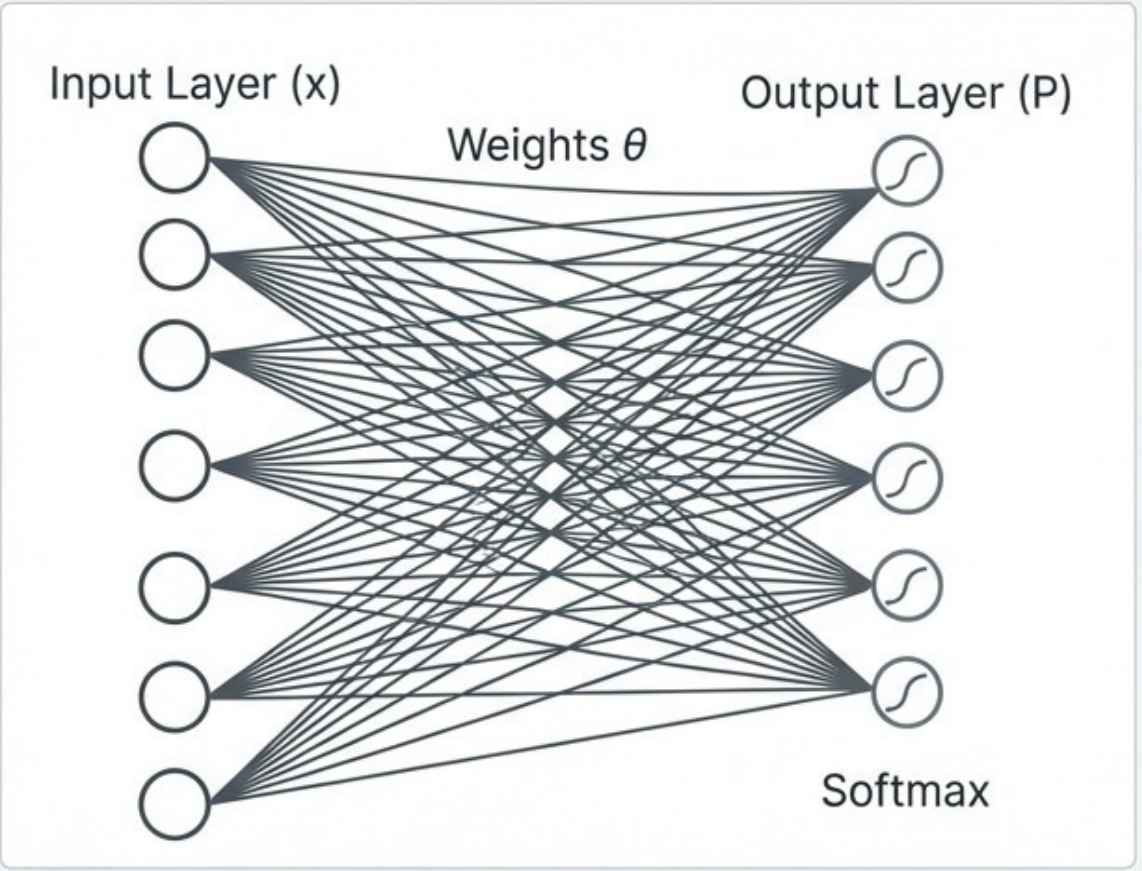
NotebookLM

The Architectural Shift

Re-imagining Softmax Regression as a Neural Network

We can visualize the regression equation $P = \text{Softmax}(Wx + b)$ as a network graph.

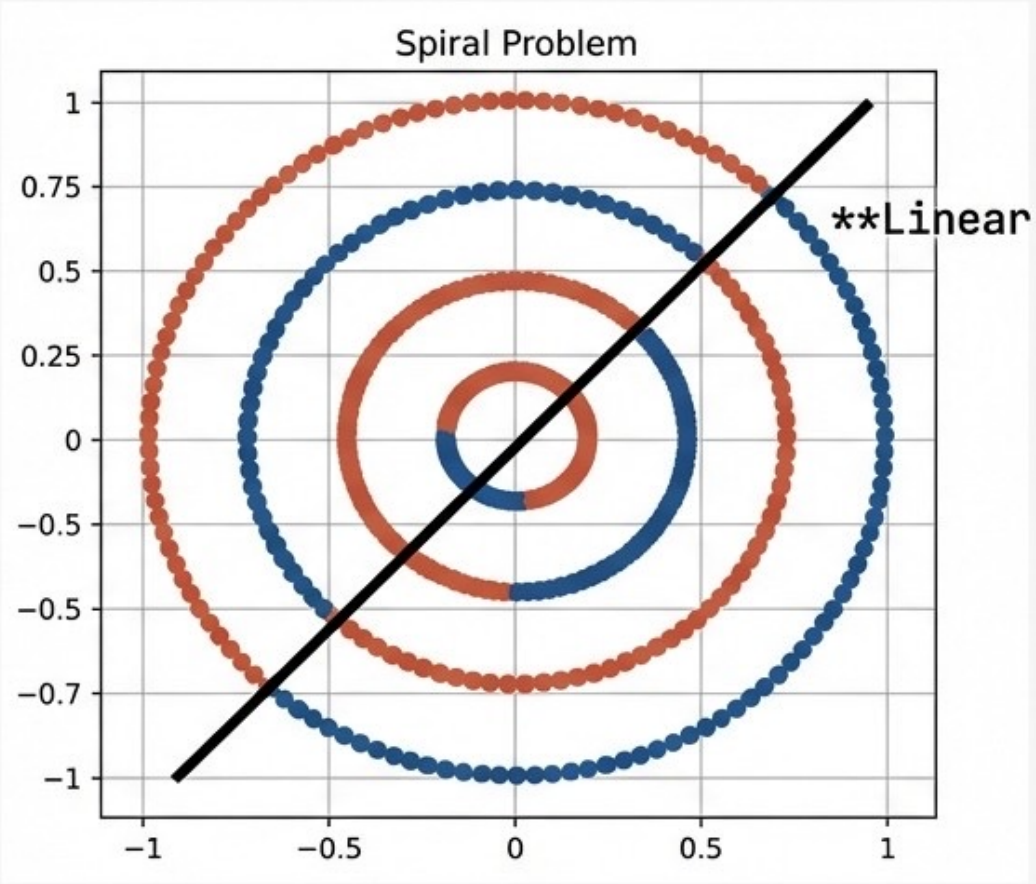
- **Input Layer**: Raw data (x)
- **Edges**: Learnable Weights (θ)
- **Output Layer**: Probability distribution (P)



NotebookLM

The Limitation: Linearity is Not Enough

A 1-layer network corresponds to a linear classifier. It separates classes using straight hyperplanes. This fails when data manifolds are entangled.



Failure Case: The Spiral Dataset is not linearly separable.

NotebookLM

The Solution: Representation Learning

We do not discard the linear classifier. Instead, we transform the input space.

$$x \rightarrow z = f(x)$$

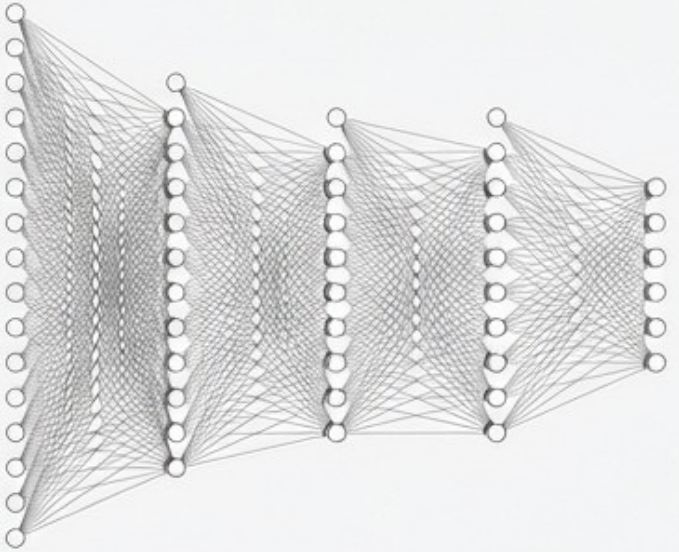


By transforming the coordinate system (Cartesian to Polar), entangled data becomes linearly separable.

NotebookLM

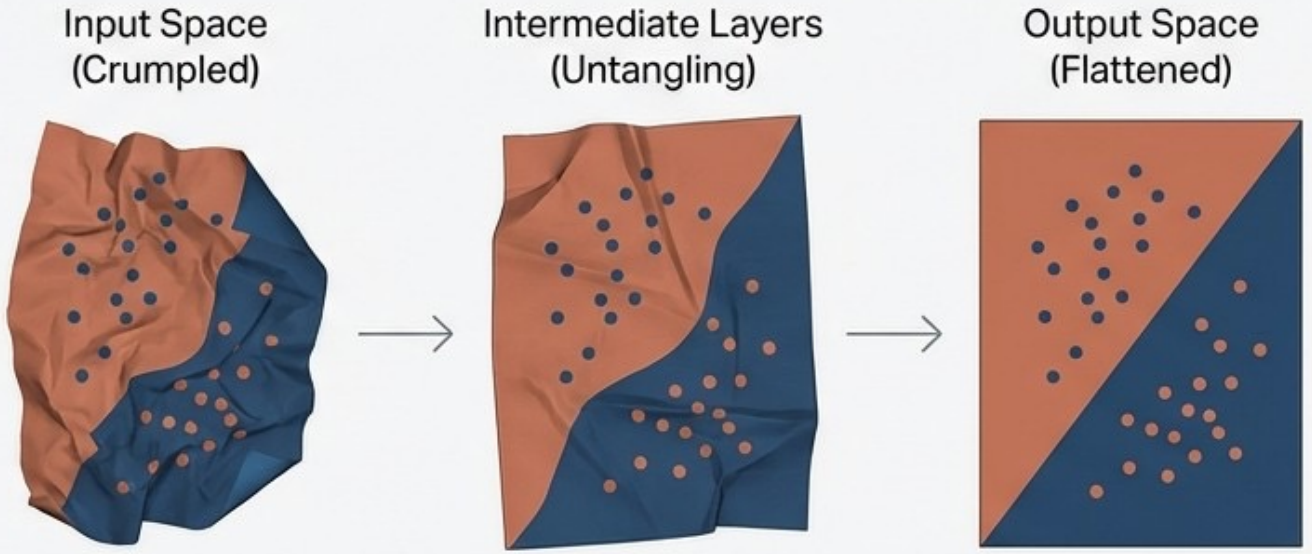
Deep Neural Networks (DNNs)

Stacking layers to disentangle manifolds.



Deep Learning generalizes the encoder concept by stacking multiple non-linear layers.

Each layer stretches and squashes the data space, progressively untangling the class manifolds until they are linearly separable at the final layer.



NotebookLM

Academic Editorial

Universal Approximation Theorem

Theorem (Hornik, 1991): A single hidden layer neural network with sufficient units can approximate *any* continuous function arbitrarily well.

Reality Check

The Nuance

1. **Representability ≠ Learnability:** Just because a solution exists doesn't mean Gradient Descent can find it.
2. **Efficiency:** Deep networks (Depth > Width) are exponentially more efficient at representing complex functions than massive shallow networks.

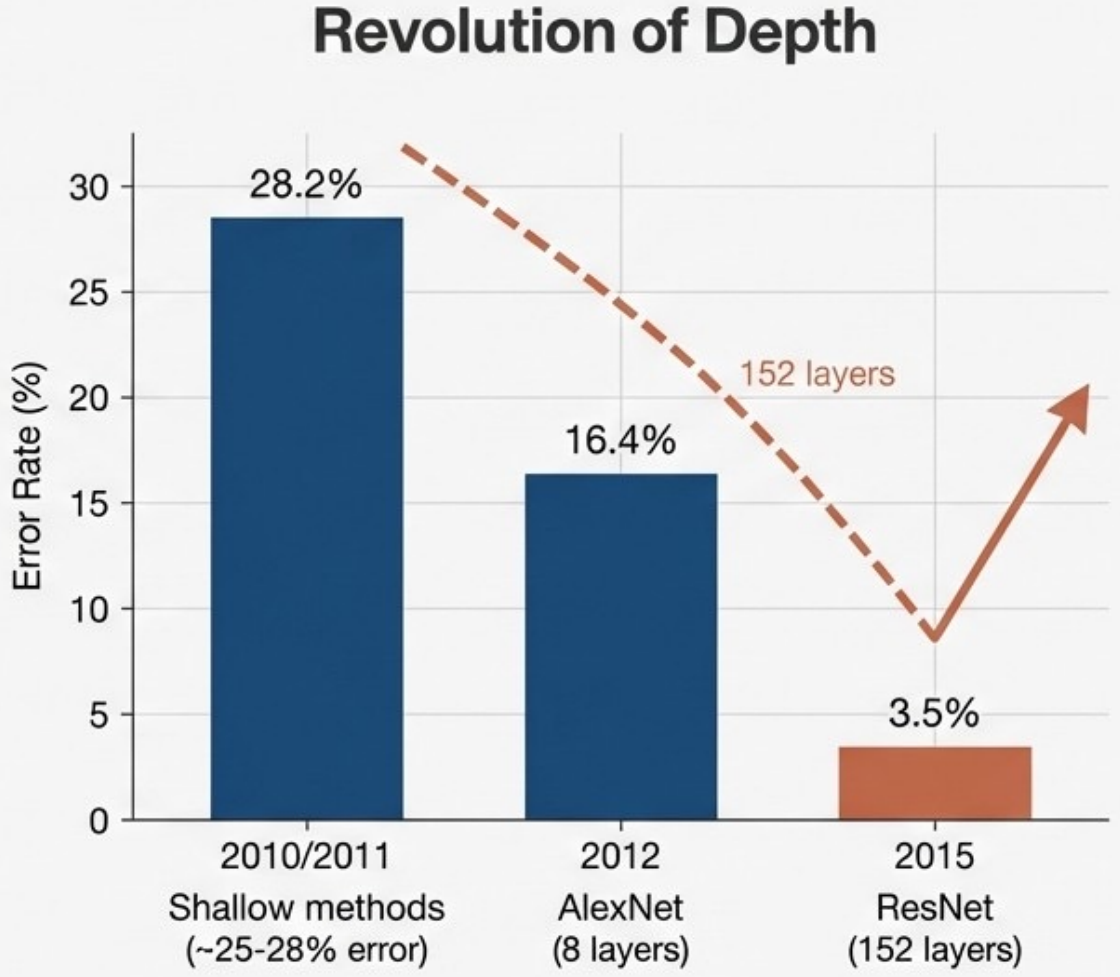
NotebookLM

The Deep Learning Revolution

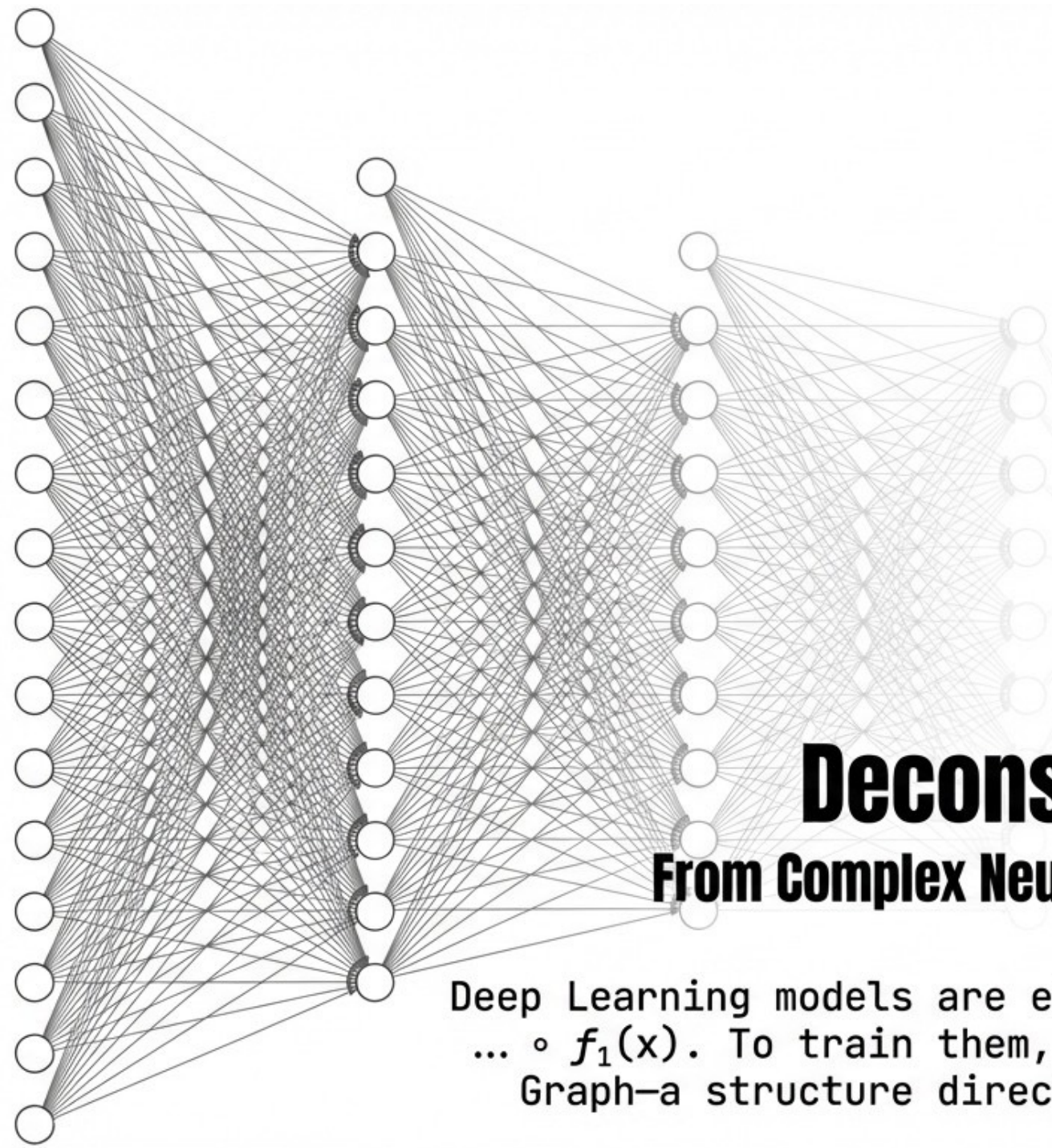
Why now? The Trinity of Success.

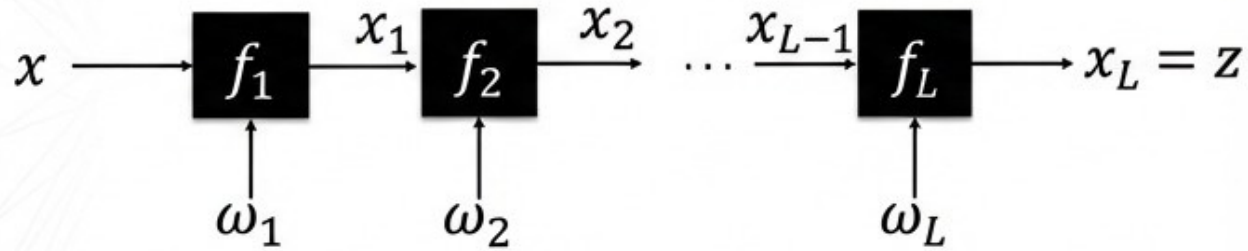
in Sabon Next, Slate Gray.

1. **Big Data:** ImageNet (1M+ labeled images).
2. **Compute:** GPU acceleration.
3. **Algorithms:** `ResNet`, `ReLU`, `Backprop`.



NotebookLM





Deconstructing Backpropagation

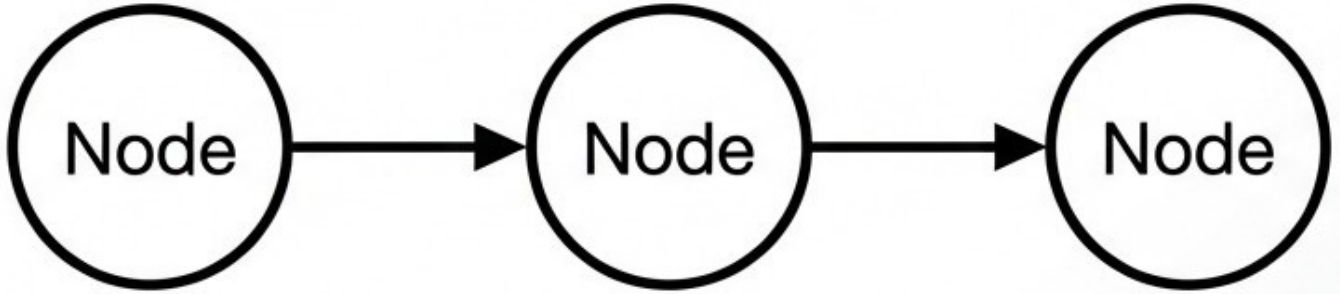
From Complex Neural Architectures to Computational Graphs

Deep Learning models are effectively composite functions: $f(x) = f_L \circ \dots \circ f_1(x)$. To train them, we map these functions as a Computational Graph—a structure directing data forward and gradients backward.

NotebookLM

THE FORWARD PASS

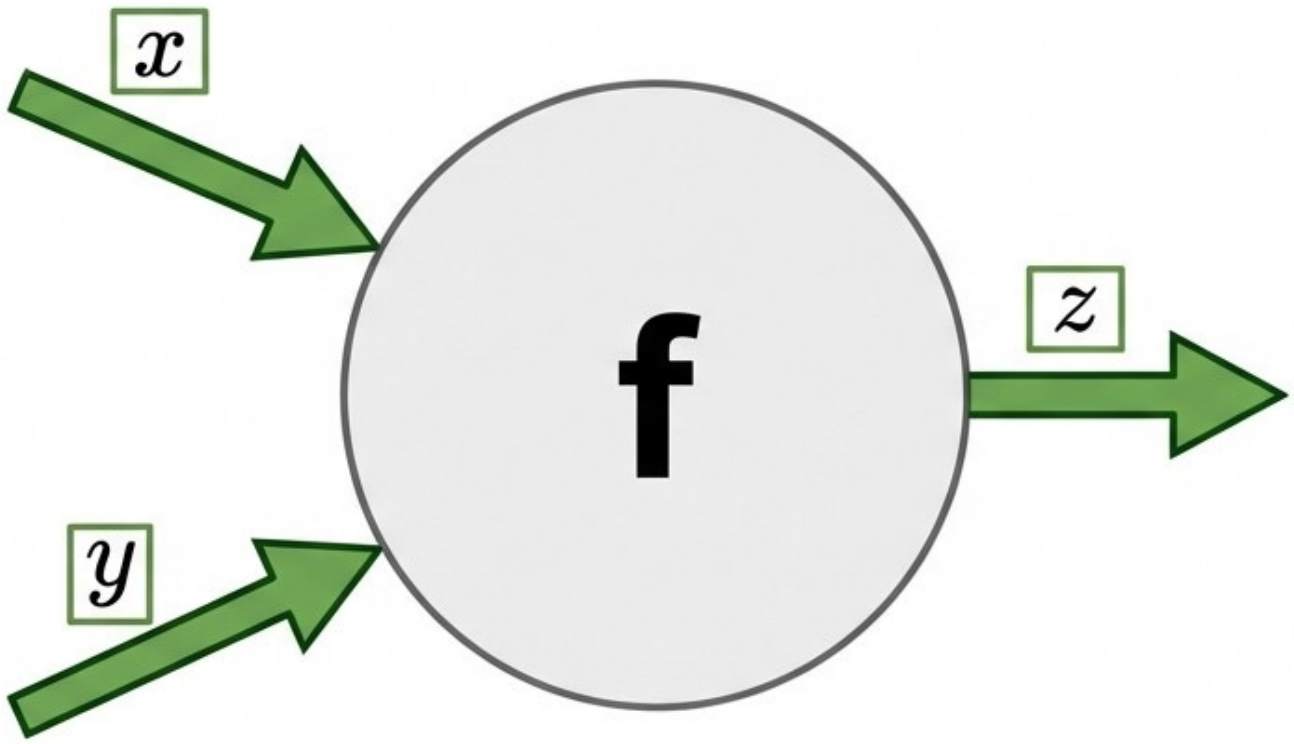
- **Input:** Data (x)
- **Action:** Compute outputs & Cache values
- **Result:** Loss score



THE BACKWARD PASS

- **Input:** Gradient of Loss ($\partial L/\partial z$)
- **Action:** Apply Chain Rule recursively
- **Result:** Parameter Updates (W)

NotebookLM

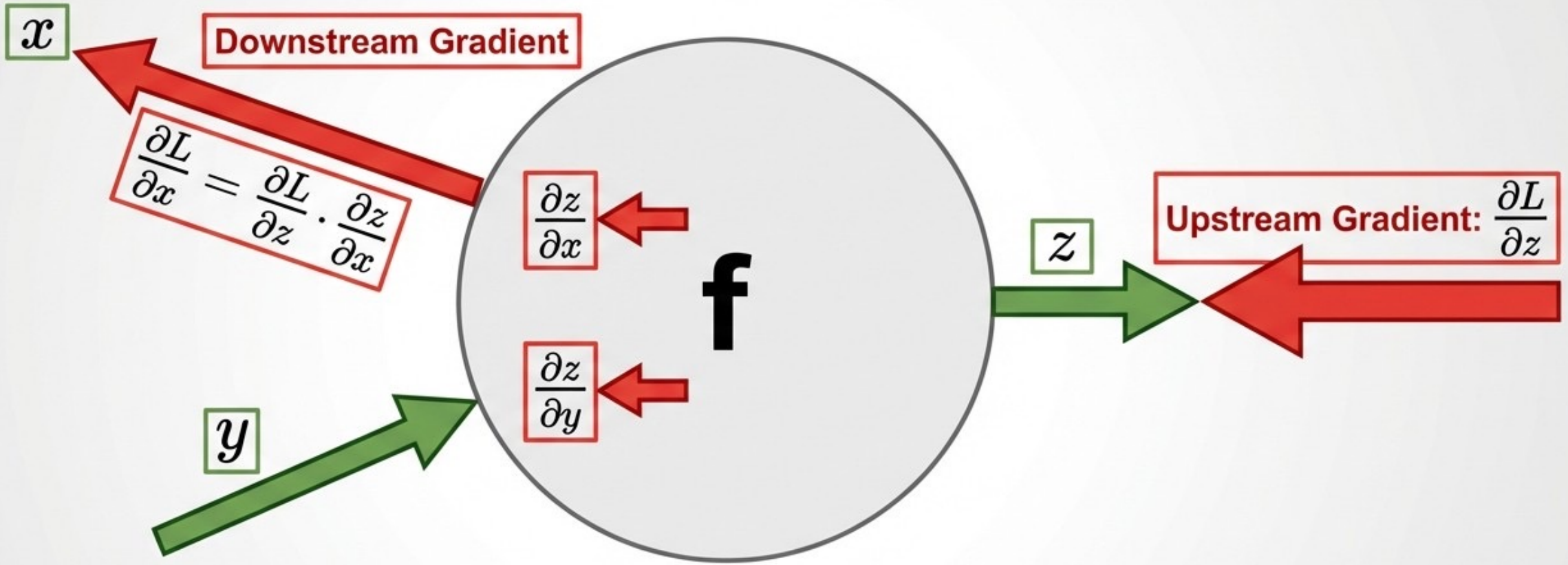


The Fundamental Unit: A Single Gate

- Regardless of network size, every operation can be isolated as a single Gate.
- The Goal: Determine how changes in inputs x and y affect the final Loss ($\mathcal{L}$).
- The Problem: The gate only knows its local inputs and outputs. It is blind to the rest of the graph.

NotebookLM

THE BACKWARD PASS: APPLYING THE CHAIN RULE

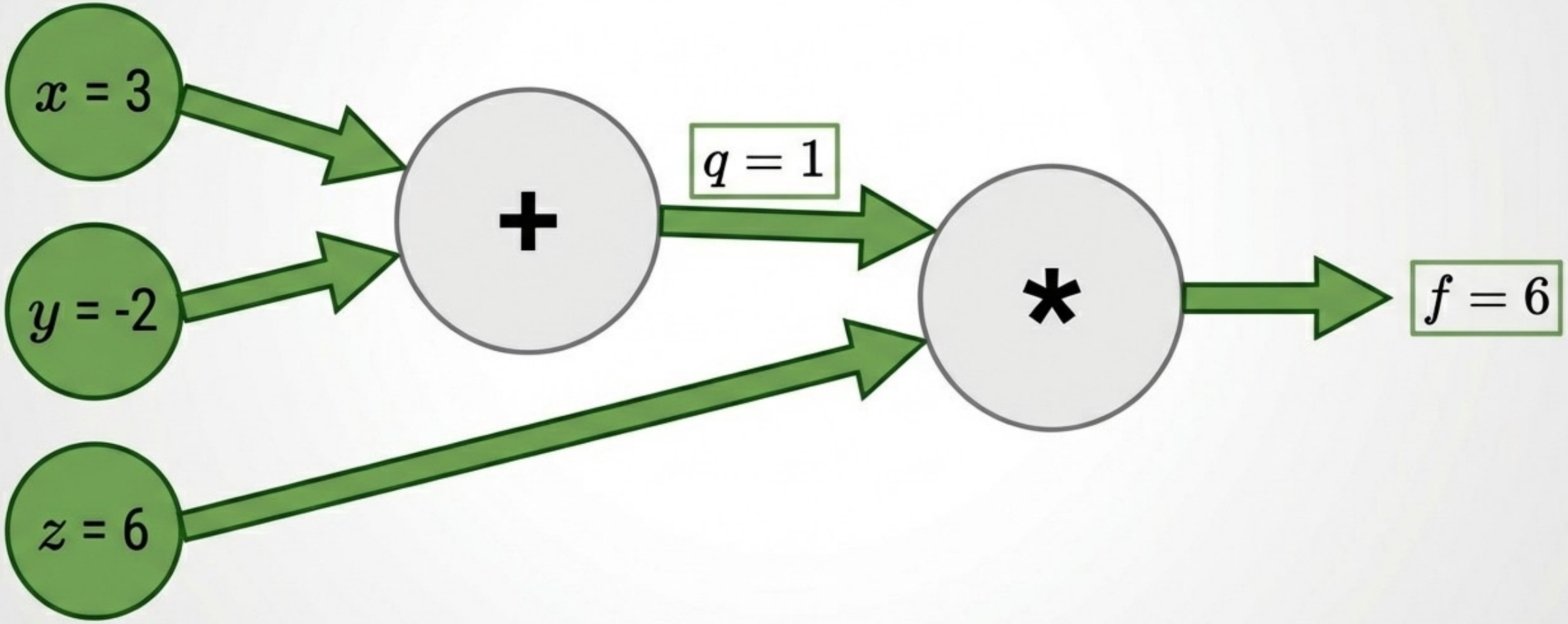


Downstream Gradient = Upstream Gradient × Local Gradient

NotebookLM

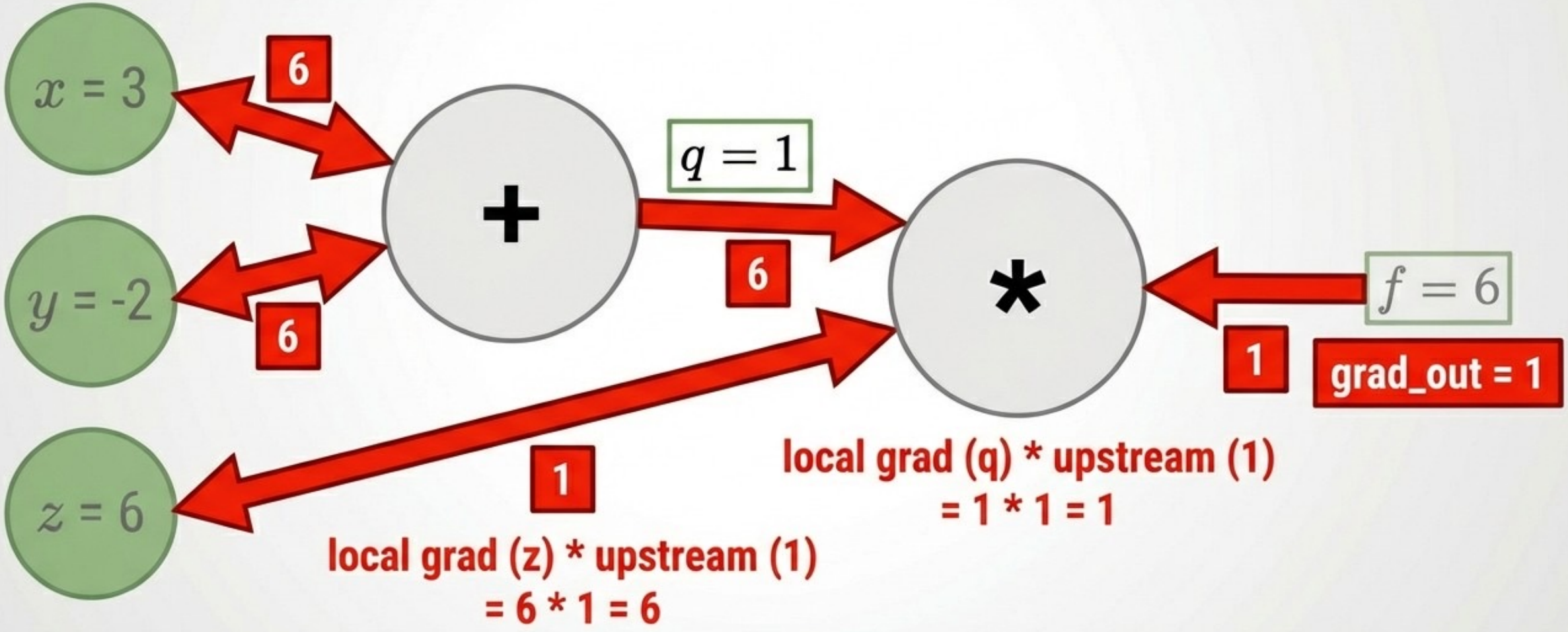
Tracing the Forward Pass

We cache the values (1, 6) in memory. We will need them later.



NotebookLM

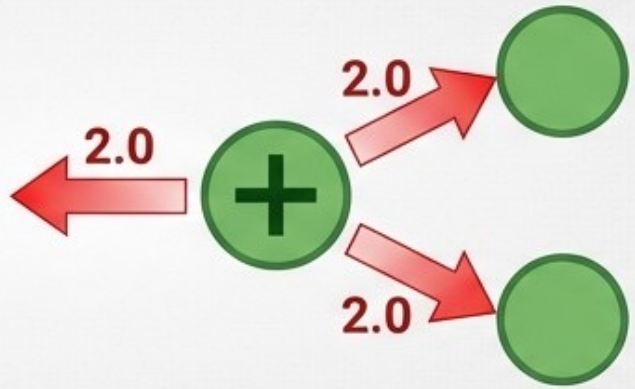
Tracing the Backward Pass



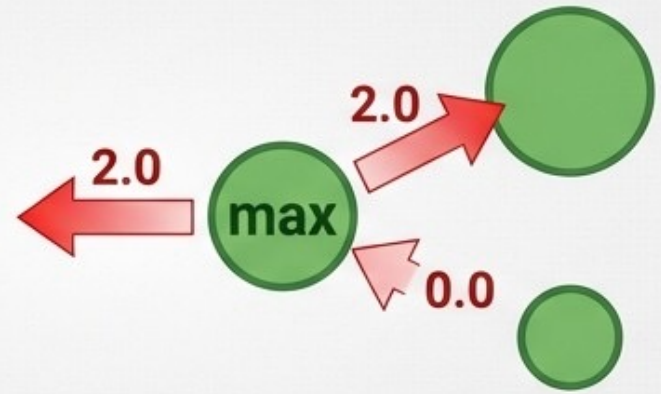
NotebookLM

Patterns in Backward Flow

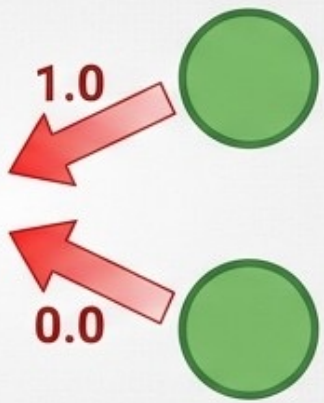
ADD Gate (+)
The Gradient Distributor.

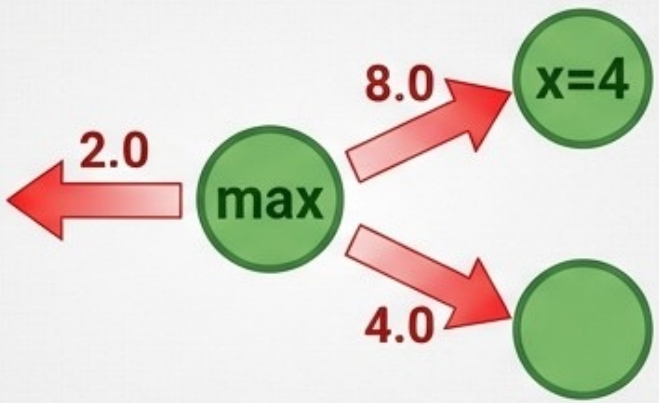


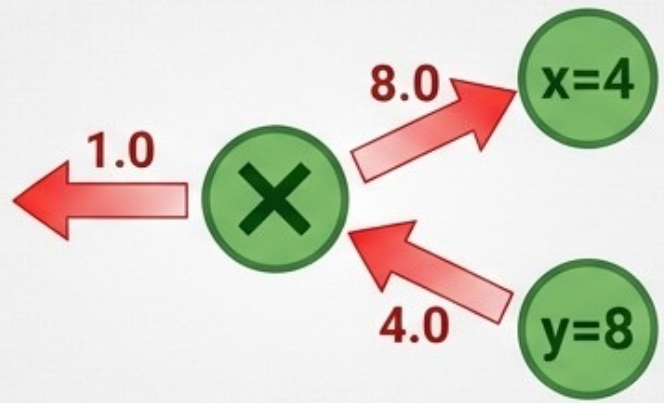
MAX Gate
The Gradient Router.



MUL Gate (*)
The Gradient Switcher.

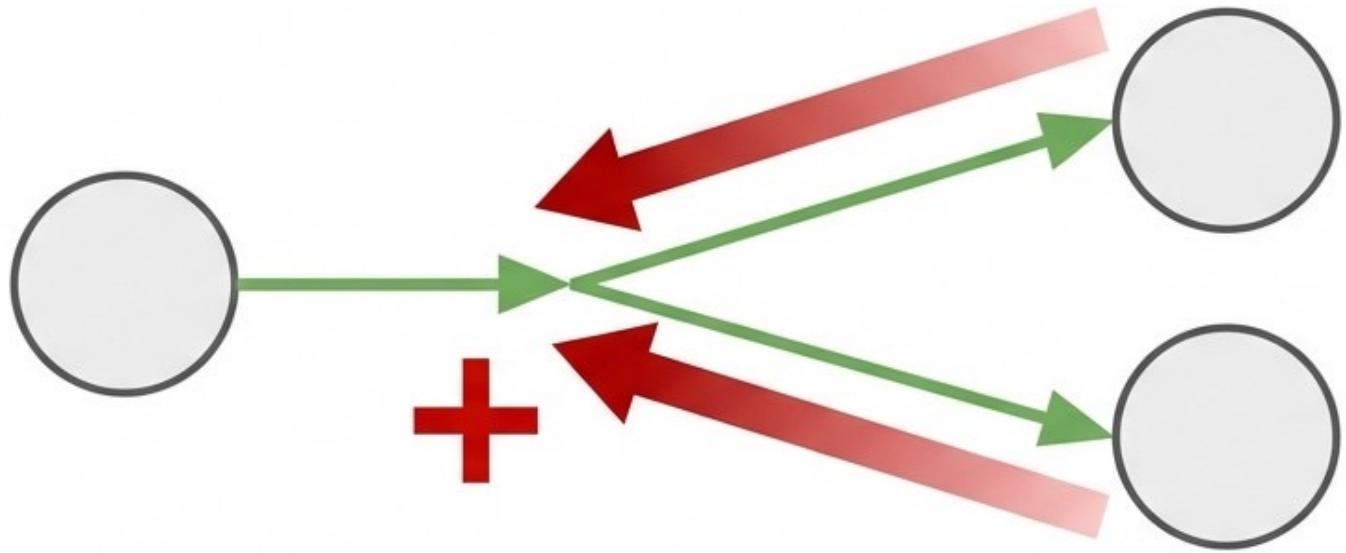






NotebookLM

The Accumulation Principle



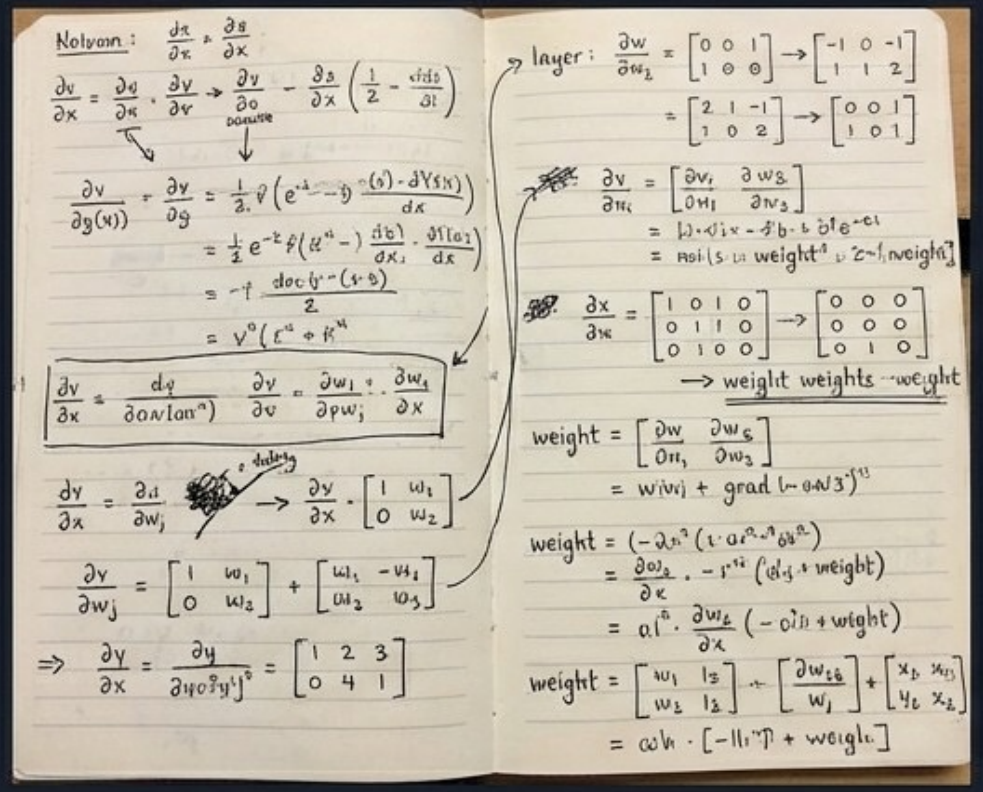
When a variable branches to different parts of the graph, its effect on the Loss is the SUM of its effects through each branch.

```
dw += gradient  # Accumulate gradients, never overwrite
```

NotebookLM

The Era of Automatic Differentiation

Manual Backprop (The Hard Way)



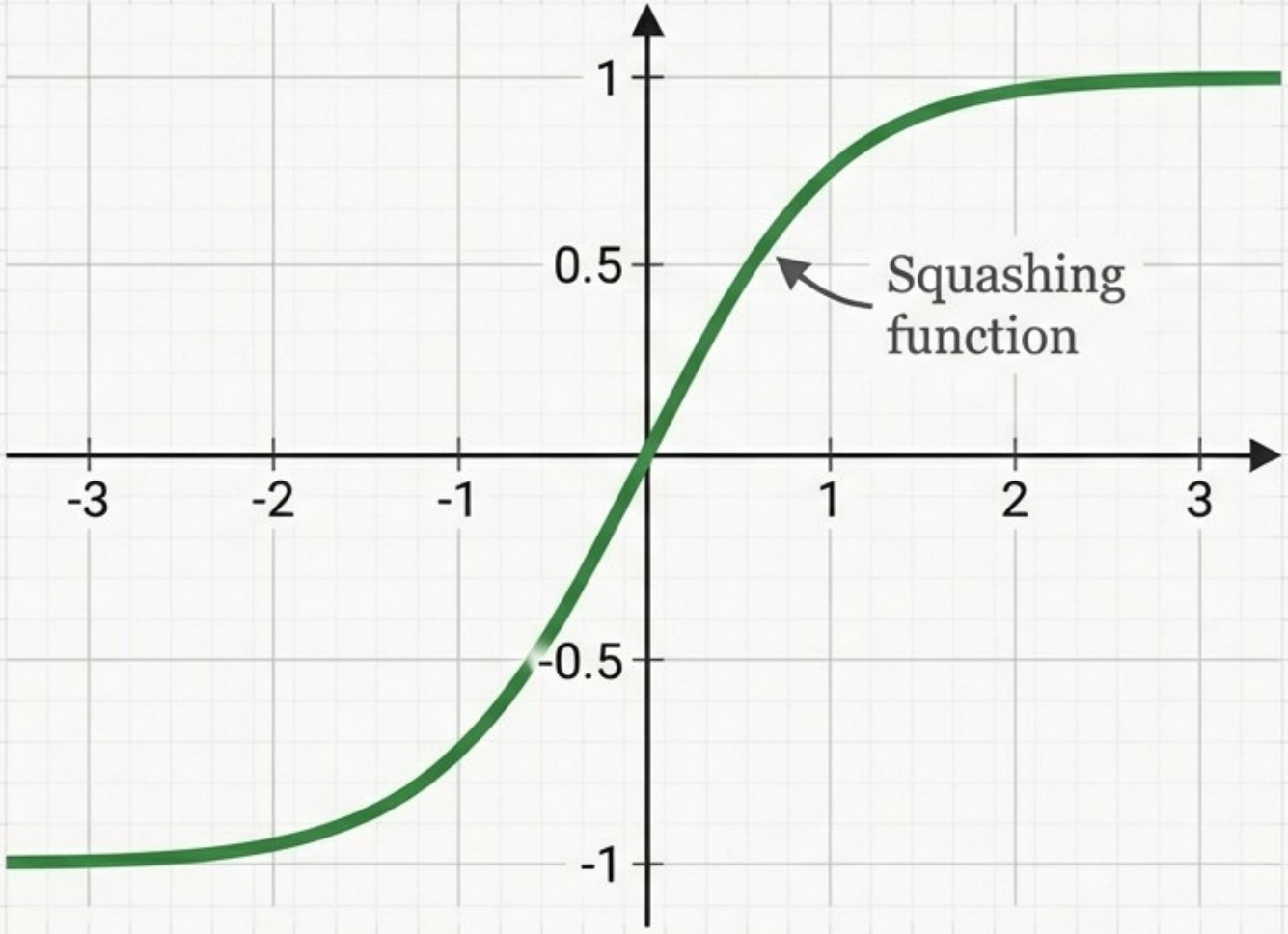
AutoDiff (The Modern Way)

```
x = tensor([1.0], requires_grad=True)
y = x ** 2
y.backward()
print(x.grad)
```

Frameworks like PyTorch build the graph dynamically and execute the backward pass for you.

NotebookLM

Activation Functions: Adding Non-Linearity



Real neurons have “firing rates”. They don’t output infinite energy.

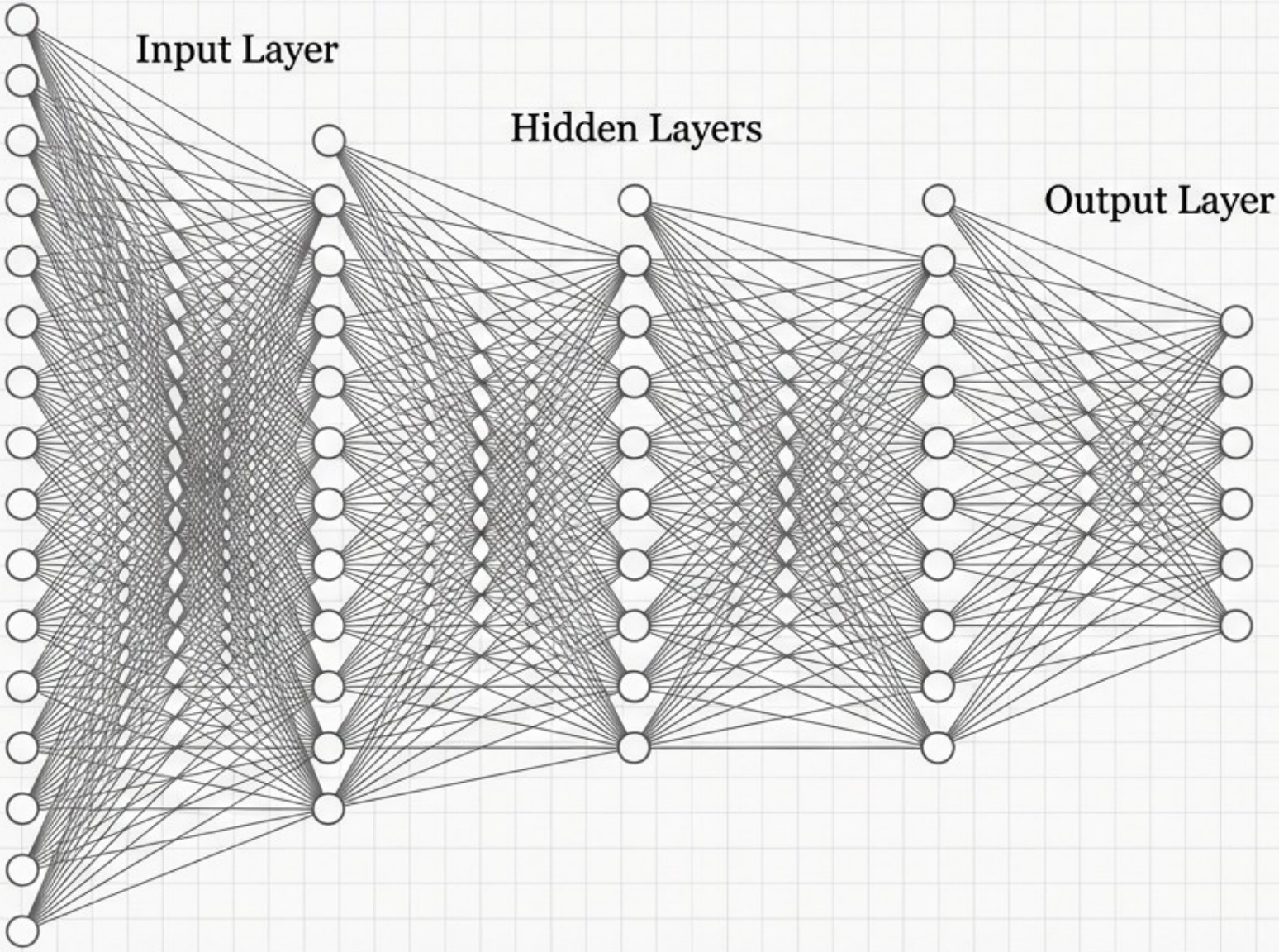
Tanh squashes inputs to the range [-1, 1].

This non-linearity allows the network to learn complex, curved boundaries.

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$$

NotebookLM

From Neuron to Multi-Layer Perceptron (MLP)



Strength in Numbers:

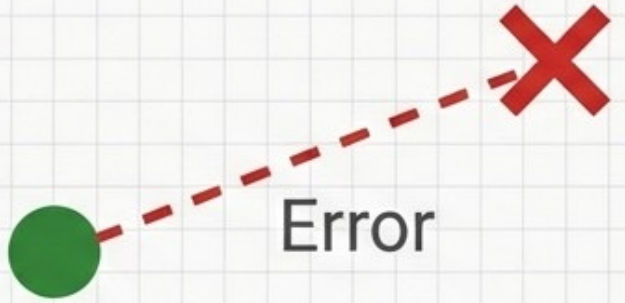
- **Layer:** A list of independent Neurons.
- **MLP:** A sequence of Layers.
- **Fully Connected:** Every neuron connects to every neuron in the next layer.

NotebookLM

The Loss Function & Optimization Loop

The Metric of Badness

$$\text{Loss} = \sum (y_pred - y_target)^2$$



The Training Loop

```
for step in range(steps):
    # 1. Forward
    ypred = model(x)
    loss = (ypred - ytarget)**2

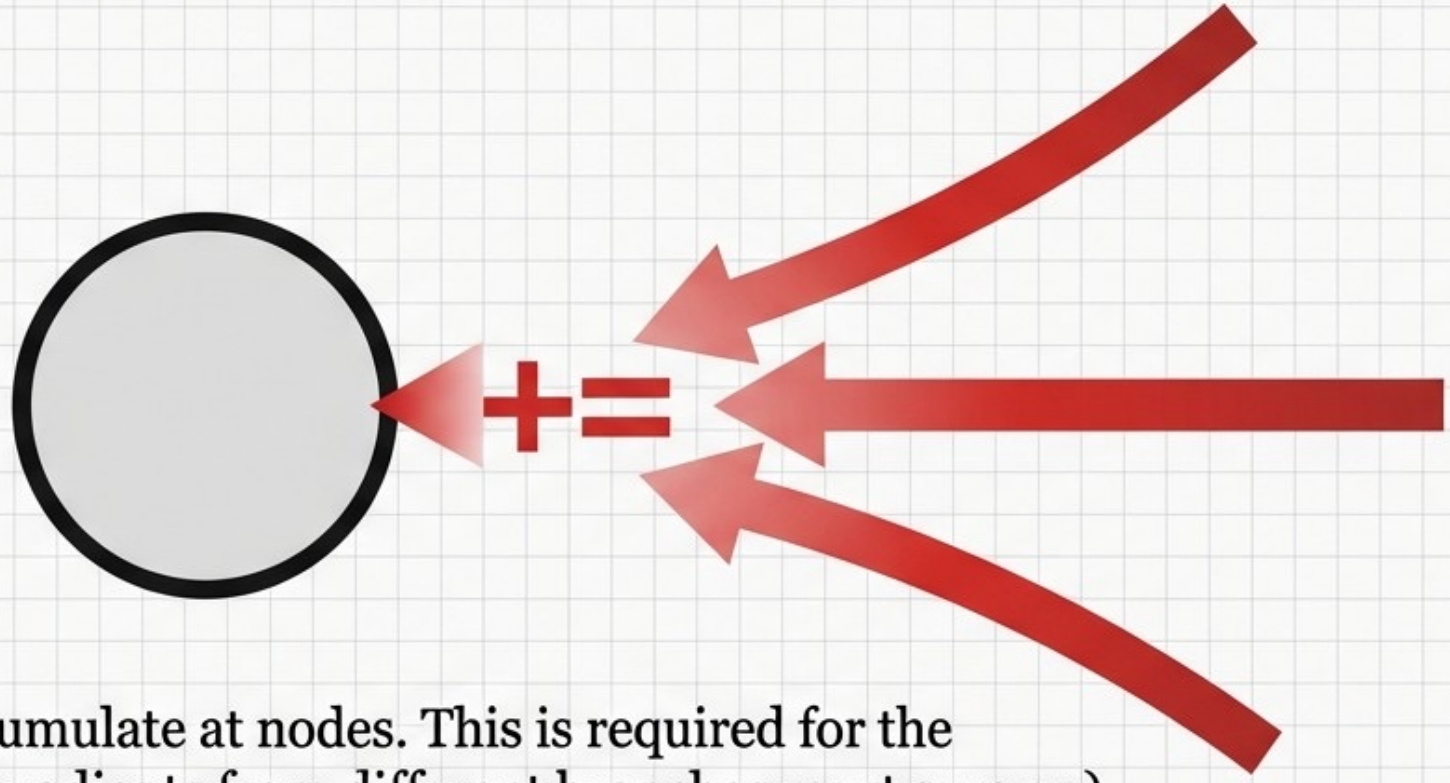
    # 2. Zero Grad
    model.zero_grad()

# Graphite Grey
# 3. Backward
loss.backward()

# 4. Update
for p in model.parameters():
    p.data += -learning_rate * p.grad
```

NotebookLM

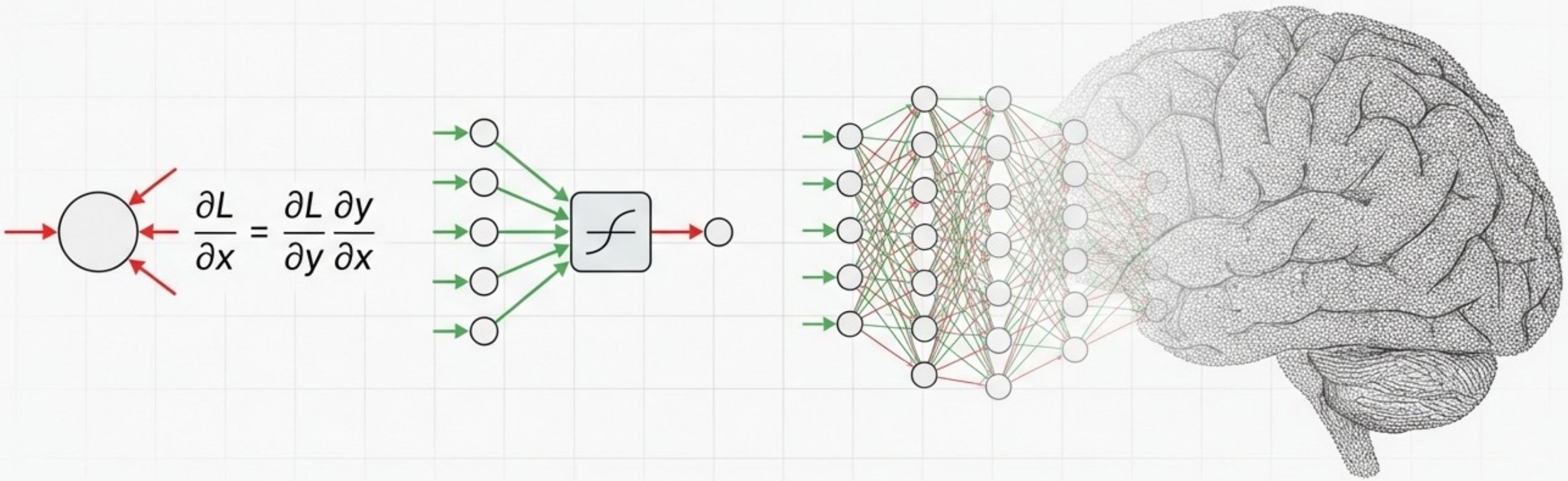
The Zero Grad Trap



The Bug: Gradients accumulate at nodes. This is required for the multivariate chain rule (gradients from different branches must sum up).
The Consequence: If you don't reset gradients to zero before the next training step, they pile up endlessly.
The Fix: Always call `zero_grad()` before `.backward()`.

NotebookLM

Emergence



From simple rules comes complex behavior. We started with a single floating point number and the Chain Rule. Scaled up by millions, this exact same logic powers GPT and modern AI. It is not magic. It is gradients.

NotebookLM